

Transducer-based linear combination of unitaries: theory and applications

Dong An^{*} Dekuan Dong[†] Changpeng Shao[‡] Yuxin Zhang[§] Chenhao Zhao[¶]

October 1, 2026

Abstract

Linear combination of unitaries (LCU) is a fundamental primitive in quantum algorithms, whose cost is typically governed by the most expensive unitary appearing in the combination. We develop a transducer-based LCU framework that reduces this worst-case dependence to a weighted average query complexity, when the constituent unitaries share access to a common set of primitive oracles.

Consider $A = \sum_j c_j U_j$, where $c_j > 0$ and each unitary U_j can be implemented using C_j primitive queries. Given an upper bound $a \geq \|A\|$, our algorithm implements a block-encoding of A/α with rescaling factor $\alpha = \mathcal{O}(a)$, using $\tilde{\mathcal{O}}(C_{\max} + \bar{C}\lambda/a)$ primitive queries, where $\lambda = \sum_j c_j$, $C_{\max} = \max_j C_j$, and $\bar{C} = \sum_j c_j C_j / \lambda$. By comparison, the standard LCU construction requires $\tilde{\mathcal{O}}(C_{\max}\lambda/a)$ primitive queries. The improvement can therefore be substantial when costly unitaries have small weights and λ/a is large. As applications, we obtain improved block-encodings of sparse matrices, leading to quantum algorithms for sparse Hamiltonian simulation and quantum linear systems with near-optimal query complexity up to polylogarithmic factors in all relevant parameters.

Our main technique is the transducer framework developed by Belovs, Jeffery, and Yalcu [Quantum, 8:1444 (2024)]. Here we develop a complementary operator-level theory tailored to block-encodings. We identify the resolvent norm K as a key complexity measure for transducer implementation besides the existing catalyst complexity. We show that the transducer action can be converted into an ϵ -approximate block-encoding using only $\mathcal{O}(K \log(1/\epsilon))$ queries to the transducer, improving the precision dependence from polynomial to logarithmic.

Contents

1	Introduction	2
1.1	Results	3
1.1.1	Main result	3
1.1.2	Applications	4
1.2	Overview of the algorithm	5
1.3	Related work and comparison	6
1.4	Organization	9

^{*}Beijing International Center for Mathematical Research, Peking University. dongan@pku.edu.cn.

[†]Beijing International Center for Mathematical Research, Peking University. ddk@pku.edu.cn.

[‡]Academy of Mathematics and Systems Science, Chinese Academy of Sciences. changpeng.shao@amss.ac.cn.

[§]Alfréd Rényi Institute of Mathematics. zhang.yuxin@renyi.hu.

[¶]School of Mathematical Sciences, Peking University. 2601110059@stu.pku.edu.cn.

2 Preliminaries	9
2.1 Problem setting	9
2.2 Quantum linear algebra primitives	11
2.3 Transducer	11
2.4 Schur-complement interpretation of transducers	13
3 Transducer toolkit	15
3.1 Transducers for composition	15
3.2 Catalyst removal via finite reuse	21
4 Transducer-based linear combination of unitaries algorithm	25
4.1 Cayley transformation of LCU with normalized coefficients	25
4.2 Algorithm description	29
4.3 Complexity analysis	30
5 Applications to sparse matrix functions	31
5.1 Sparse matrix block-encoding	32
5.1.1 A review of the standard sparse block-encoding with amplification	32
5.1.2 Matrix decomposition	32
5.1.3 Complexity	34
5.2 Sparse Hamiltonian simulation	35
5.3 Sparse linear system algorithm	35
6 Conclusion and outlook	36
A Polynomial approximation of the inverse Cayley transformation and amplification	40
B A simple explanation of the construction in Lemma 3.2	41

1 Introduction

A fundamental challenge in quantum algorithm design is to harness unitary quantum dynamics to manipulate the broad class of non-unitary operators that arise across a wide range of scientific applications. Linear combination of unitaries (LCU) provides a simple and remarkably versatile solution to this problem by embedding such operators into larger unitary transformations, and has consequently become a basic primitive underlying a wide range of quantum algorithms, including quantum linear system solvers [1], Hamiltonian simulation [2, 3], differential equations [4–6], quantum walks [7, 8], ground state preparation [9–11], quantum eigenvalue transformations [12, 13], etc. It also serves as one of the basic building tools for constructing block-encodings used in quantum singular value transformation (QSVT) [14], a general framework for polynomial transformations of matrices.

The basic LCU construction [15] is conceptually simple. Suppose that $A = \sum_j c_j U_j$, where $c_j > 0$ and each U_j is an efficiently implementable unitary, and let $\lambda = \sum_j c_j$. One first prepares an ancilla state whose amplitudes encode the square root of the coefficients, $\sqrt{c_j}$, and then applies a controlled selection operation (commonly called the select oracle) $\sum_j |j\rangle \langle j| \otimes U_j$. Uncomputing the ancilla yields a larger unitary whose appropriate block is exactly A/λ . In this way, a generally non-unitary operator A is embedded into a unitary quantum circuit and can subsequently be manipulated using standard quantum algorithmic primitives.

The efficiency of this construction, however, is determined not only by the normalization factor λ , but also by the cost of implementing the controlled selection operation. To be more exact, suppose each U_j costs C_j and under reasonable model assumptions, $\sum_j |j\rangle \langle j| \otimes U_j$ can

be implemented in cost $\mathcal{O}(C_{\max})$ with $C_{\max} = \max_j C_j$.¹ Given a known upper bound $a \geq \|A\|$ with $0 < a \leq \lambda$, standard LCU followed by singular value amplification [14] constructs an $\mathcal{O}(a)$ -normalized block-encoding at a query cost of $\tilde{\mathcal{O}}(C_{\max}\lambda/a)$.

One may realize that the coefficients of an LCU naturally define a probability distribution over its constituent operations. Indeed, writing $p_j = c_j/\lambda$, a classical randomized procedure that selects the j -th operator with probability p_j would lead to the expectation A/λ , incurring the expected query cost $\bar{C} = \sum_j p_j C_j$. Thus LCU carries an intrinsic weighted-average notion of complexity. This average-cost structure is, however, largely lost in the standard coherent implementation of LCU. The select operation must implement all constituent operations coherently and is therefore typically governed by the worst-case cost $C_{\max}$. A natural question is whether one can retain quantum coherence while recovering the weighted-average cost inherent in the LCU decomposition.

In this work, we identify a general weighted-cost principle for LCU by developing a transducer-based LCU framework. Specifically, we reduce the standard LCU cost from $\tilde{\mathcal{O}}(C_{\max}\lambda/a)$ to $\tilde{\mathcal{O}}(C_{\max} + \bar{C}\lambda/a)$. Therefore our approach can be advantageous when the high-cost unitaries have small coefficient weights and the coefficient 1-norm is much larger than the spectral norm of the matrix A . As applications, we obtain an improved block-encoding of sparse matrices, which in turn yields quantum algorithms for sparse Hamiltonian simulation and sparse linear systems with query complexities matching the lower bounds of [16–18] up to polylogarithmic factors.

1.1 Results

1.1.1 Main result

Our main result gives a general LCU construction whose normalization can be reduced at a cost determined by the weighted average of the constituent query complexities.

Result 1.1 (Transducer-based LCU, informal version of Theorem 4.2). *Let U_j be unitaries that can be implemented by C_j queries to a fixed set of primitive oracles. Let $A = \sum_{j=0}^{J-1} c_j U_j$ with coefficients $c_j > 0$, and suppose that an upper bound $a \geq \|A\|$ is known. Then, there exists an algorithm that implements A/α with $\alpha = \mathcal{O}(a)$ up to error $\mathcal{O}(\epsilon)$, using*

$$\mathcal{O}\left(C_{\max} \log\left(\frac{1}{\epsilon}\right) + \frac{\lambda}{a} \bar{C} \left(\log\left(\frac{1}{\epsilon}\right)\right)^2\right) \quad (1)$$

queries to the primitive oracles, where $\lambda = \sum_{j=0}^{J-1} c_j$, $C_{\max} = \max_j C_j$, $\bar{C} = (\sum_{j=0}^{J-1} c_j C_j)/\lambda$.

Result 1.1 requires two conditions. First, the circuits for U_j 's are related in the sense that their oracle calls belong to a common family that can be selected coherently (called shared primitive access assumption, as specified in Assumption 2.1). While excluding the most general case when all the unitaries are unrelated, this condition can be satisfied in many practical scenarios, such as the quantum phase estimation circuit, and the Hamiltonian simulation and linear system algorithms that we will discuss in Section 1.1.2. Second, an upper bound of $\|A\|$ should be known a priori. Although we may simply take $a = \lambda$, such a choice cannot lead to a genuinely improved query complexity as being discussed in Section 1.3.

In fact, the main improvement concerns the cost of reducing the normalization from λ to $\mathcal{O}(a)$. In the standard construction of LCU, it requires repeated uses of the entire select oracle, each costing the most expensive complexity $C_{\max}$. In our algorithm, the repeated cost is the averaged cost $\bar{C}$, while $C_{\max}$ appears only as an additive contribution. The saving can therefore be substantial when expensive unitaries carry small coefficient weights and λ/a is large. The

¹But generally, the cost is $\sum_j C_j$, here we assume it is dominated by the worst unitary for simplicity. This is also the common scenario in practice.

additive term also makes clear that our result does not imply that an arbitrary constituent circuit can be implemented at its average cost.

Our Result 1.1 applies more generally to $A = \sum_{j=0}^{J-1} c_j A_j$, where A_j are general matrices supplied through their block-encodings. This general formulation allows our algorithm to be used as a subroutine in other algorithms based on block-encodings, such as the sparse matrix calculations considered in this paper.

1.1.2 Applications

We apply our transducer-based LCU to construct a block-encoding of a sparse matrix, which then serves as the common input model for sparse matrix functions such as Hamiltonian simulation and quantum linear system solving. We use the standard sparse-access model, with oracles that return the locations and values of nonzero entries. For the applications below, we consider Hermitian matrices with unit spectral norm.

Result 1.2 (Block-encoding of a sparse matrix, informal version of Corollary 5.4). *A block-encoding of a d -sparse normalized matrix can be constructed with $\mathcal{O}(1)$ rescaling factor and $\mathcal{O}(\sqrt{d} \text{polylog}(d/\epsilon))$ queries to its standard sparse access oracles.*

Result 1.2 gives a block-encoding whose rescaling factor remains constant while the number of sparse oracle queries grows only as $\sqrt{d}$, up to poly-logarithmic factors. A constant rescaling factor is useful because it avoids introducing an additional sparsity dependence into subsequent matrix transformations. The construction therefore provides a reusable representation of the matrix, with the query savings available to any algorithm that uses this block-encoding.

Our first application of this representation is Hamiltonian simulation. For a normalized Hamiltonian A , the goal is to implement e^{-iAT} for a time T . Combining the sparse block-encoding with quantum singular value transformation (QSVT) [14] yields nearly linear dependence on the evolution time T , with each use of the block-encoding costing $\tilde{\mathcal{O}}(\sqrt{d})$ sparse oracle queries.

Result 1.3 (Near-optimal sparse Hamiltonian simulation algorithm, informal version of Theorem 5.5). *There exists a quantum algorithm which simulates a d -sparse normalized Hamiltonian to time T with error at most $\mathcal{O}(\epsilon)$, using $\mathcal{O}(\sqrt{d}T \text{polylog}(dT/\epsilon))$ queries to the Hamiltonian sparse oracle.*

For a general Hamiltonian, a known spectral norm bound can be absorbed into the evolution time by rescaling. The leading $\sqrt{d}T$ dependence matches the worst-case lower bound of [16] up to polylogarithmic factors. Thus Result 1.3 is near-optimal in its joint dependence on sparsity and time, and the accuracy dependence achieves a polylogarithmic scaling.

The same block-encoding can also be combined with quantum linear system solvers. Given an invertible d -sparse Hermitian matrix A with $\|A\| = 1$, condition number $\kappa = \|A\|\|A^{-1}\|$, and a normalized right-hand-side state $|b\rangle$, the goal is to prepare the normalized solution state $A^{-1}|b\rangle / \|A^{-1}|b\rangle\|$.

Result 1.4 (Near-optimal sparse linear system algorithm, informal version of Theorem 5.6). *There exists a quantum algorithm which solves the quantum sparse linear system problem $Ax = |b\rangle$ with error at most $\mathcal{O}(\epsilon)$, using $\mathcal{O}(\sqrt{d}\kappa \text{polylog}(d\kappa/\epsilon))$ queries to the matrix sparse oracle and $\mathcal{O}(\kappa/\|A^{-1}|b\rangle\|)$ queries to the right-hand-side state.*

In Result 1.4, the matrix-query bound has leading dependence $\sqrt{d}\kappa \text{polylog}(1/\epsilon)$, matching the joint lower bound of [18] in all parameters up to polylogarithmic factors. This dependence can follow from using our block-encoding in any existing quantum linear system solver whose matrix-query cost is nearly linear in κ and poly-logarithmic in $1/\epsilon$. Here we use the tunable variable-time amplitude amplification (tunable VTAA) algorithm of [19], which not only yields

the desired matrix query complexity but also gives the $\mathcal{O}(\kappa/\|A^{-1}|b\rangle\|)$ scaling for the number of preparing $|b\rangle$, matching the lower bound for the state preparation cost also established in [19]. The ratio $\kappa/\|A^{-1}|b\rangle\|$ lies between 1 and κ and can be much smaller than κ , making this distinction useful when preparing $|b\rangle$ is expensive. In summary, our sparse block-encoding improves the matrix query complexity while preserving this favorable dependence on right-hand-side preparation.

In this work, we study only two representative applications of Result 1.2 in detail. More generally, when combined with QSVT, Result 1.2 leads to improved quantum algorithms for general functions of sparse matrices, yielding a near-quadratic improvement in the dependence on sparsity compared with the algorithm of [20]. Moreover, by combining the low bound techniques of [16, 20], one can show that the square-root dependence on the sparsity is indeed optimal.

1.2 Overview of the algorithm

The guiding idea is to retain the intermediate stages of the constituent circuits throughout the LCU construction. To this end, a convenient way is through the transducer, which was proposed in [21] and recently received rapidly increasing attention in quantum algorithm design [22–26]. In a transducer, its state space has a public part, which carries the desired input and output, and a private part containing an auxiliary vector called the catalyst unchanged in the algorithm. It is an idealization of the usual bounded-error quantum algorithm that can exhibit useful properties such as efficient composition. Specifically, one can turn a circuit consisting of a sequential composition of oracles into a large unitary transducer with only a constant number of controlled oracle calls, similar to the select oracle in the standard LCU implementation. Nevertheless, the overall complexity, instead of being reduced to a constant level, is transferred to the norm of the catalyst, which may exhibit the average cost among the unitaries.

Our LCU algorithm uses this representation in three steps. For clarity, we describe the Hermitian unitary case. The usual Hermitian dilation extends the construction to general matrices, and a complete technical overview of our algorithm is given in Section 4.2.

Step 1: Combine the constituent unitaries as transducers. We first represent each unitary U_j , consisting of sequential oracle calls, by a transducer S_j whose catalyst contains its intermediate states. A single application of S_j coherently calls the oracles in parallel (like the select oracle in standard LCU). We then combine them coherently using coefficient amplitudes proportional to $\sqrt{c_j}$, so the squared size of the combined circuit histories is governed by $\overline{C}$. A simple phase operation turns the construction into a transducer whose public action is the Cayley transformation of $A/(4\lambda)$, defined as $\text{Cay}(A/(4\lambda)) = (I - iA/(4\lambda))(I + iA/(4\lambda))^{-1}$. Notice that here the rescaling factor of A is still on the order of the 1-norm of the coefficients in LCU.

Step 2: Recover the target linear combination and improve its normalization. We next design a QSVT circuit that approximately inverts the Cayley transformation and amplifies the resulting block from rescaling factor $\mathcal{O}(\lambda)$ to $\mathcal{O}(a)$. The length of such a QSVT circuit is $L = \mathcal{O}((\lambda/a) \log(1/\epsilon))$, the degree of the polynomial for approximate inverse Cayley transformation and amplification. Crucially, we do not actually implement this QSVT circuit, but assemble a transducer for this entire circuit in a similar way as we perform in Step 1. The public action of the resulting transducer is a block-encoding of $A/(16a)$, and the amplification ratio λ/a enters the complexity through the catalyst norm, which is multiplied only on the average cost.

Step 3: Convert the transducer to the standard block-encoding. The catalyst is a mathematical description of the computation and need not be prepared. In the standard procedure [21], one may instead start with zero amplitude in the private part,

distribute the input coherently among labelled public sectors, and repeatedly apply the transducer while reusing the same private space. Recombining the public sectors gives an approximate implementation of the desired action. However, the basic construction converges only algebraically with the number of repetitions, leading to polynomial dependence on $1/\epsilon$. To improve the accuracy dependence, we leverage the idea of extrapolation, which is a widely used methodology for constructing high-order numerical methods from lower-order ones in both classical numerical analysis and quantum algorithm design. Specifically, we linearly combine multiple circuits from our algorithm with different transducer repeats at the final stage to cancel low-order error terms, leading to an approximation of the block-encoding of $A/(16a)$ with exponentially reduced error.

The complexity then follows from keeping track of two different contributions. The longest constituent circuit sets an initial scale $\mathcal{O}(C_{\max})$, while composing L uses of the Cayley transformation in Step 2 adds $\mathcal{O}(L\bar{C})$. High-order extrapolation in Step 3 incurs one further factor of $\log(1/\epsilon)$. The resulting primitive query cost is therefore $\mathcal{O}((C_{\max} + L\bar{C})\log(1/\epsilon))$, which gives the claimed scaling in Result 1.1. At the technical level, the quantity controlling the final conversion is the resolvent norm of the private part of the transducer (see Theorem 3.5). Its composition bound (see Corollary 3.3) is what allows the longest circuit to contribute additively while the amplification cost accumulates with the weighted average.

Sparse matrix applications. For a sparse matrix A with $\|A\| \leq 1$, motivated by [16], we start with a useful decomposition $A = \sum_{j=0}^m A_j$ that groups matrix entries according to their magnitudes, with adjacent thresholds differing by a factor of two. Then we block-encode each component A_j by the standard sparse matrix block-encoding with amplitude amplification [16], and finally linearly combine them by our transducer-based LCU. This gives a block-encoding of the original sparse matrix A with $\mathcal{O}(1)$ rescaling factor. See Section 5.1 for a detailed description of this approach.

The overall sparsity dependence is only square root, up to polylogarithmic factors, for the following reason. In a sparse matrix, small entries can be numerous, whereas a bound on the Euclidean norm of each column limits how many large entries it can contain. This gives the tradeoff that groups containing larger entries require more queries but have smaller rescaling factors. These two effects compensate in the weighted sum of costs, which can be leveraged by our transducer-based LCU. More specifically, for a matrix of spectral norm at most one, this decomposition has $\lambda = \mathcal{O}(\sqrt{d})$, $C_{\max} = \mathcal{O}(\sqrt{d} \log d)$, and $\bar{C} = \mathcal{O}((\log d)^2)$ (See Lemma 5.2). Transducer-based LCU therefore gives a block-encoding with constant normalization using $\mathcal{O}(\sqrt{d} \text{polylog } d)$ queries.

With a block-encoding available, Hamiltonian simulation and linear system tasks can be readily solved by any existing quantum algorithms. We obtain our Result 1.3 by combining transducer-based LCU with QSVT for Hamiltonian simulation [14], and Result 1.4 by using the tunable VTAA [19]. In Section 5.3, we also discuss the complexity of combining the transducer-based LCU with the discrete adiabatic linear system solver [27], which is slightly better in the number of matrix queries but has worse state preparation cost.

1.3 Related work and comparison

Standard LCU. The usual LCU construction prepares a coefficient state, applies the select oracle, and unprepares the coefficient state [2, 15]. In the block-encoding formulation, it yields normalization λ , which can be reduced using singular value amplification [14]. Under the shared primitive access assumption, a select oracle can be implemented using $\mathcal{O}(C_{\max})$ queries. Repeating it for amplification gives a cost $\mathcal{O}(C_{\max}\lambda/a)$ for a constant error level. Our transducer-based LCU replaces this multiplicative dependence by $\mathcal{O}(C_{\max} + \bar{C}\lambda/a)$.

Nevertheless, a more careful complexity comparison must take into account both the query complexity, which directly affects the cost per run, and the rescaling factor, which determines the success probability of necessary post-selection steps. In Table 1, we compare our transducer-based LCU algorithm to the standard LCU with or without uniform singular value amplification. In particular, we additionally present an “overall query complexity”, i.e., the product of the query complexity and the rescaling factor, which can be a representative complexity estimate when applying LCU on an input state and extracting the output state with amplitude amplification. The comparison in Table 1 hereby shows that our transducer-based LCU can be advantageous when expensive unitaries are less weighted and the norm of the combined matrix is much less than the coefficient 1-norm.

Method	Query complexity per run	Rescaling	Overall query complexity
Standard LCU [15]	$\mathcal{O}(C_{\max})$	$\mathcal{O}(\lambda)$	$\mathcal{O}(\lambda C_{\max})$
Standard LCU with amplification [14]	$\mathcal{O}(C_{\max} \frac{\lambda}{a} \log(\frac{1}{\epsilon}))$	$\mathcal{O}(a)$	$\mathcal{O}(\lambda C_{\max} \log(\frac{1}{\epsilon}))$
Our work (Theorem 4.2)	$\mathcal{O}(C_{\max} \log(\frac{1}{\epsilon}) + \frac{\lambda}{a} \bar{C} (\log(\frac{1}{\epsilon}))^2)$	$\mathcal{O}(a)$	$\mathcal{O}(a C_{\max} \log(\frac{1}{\epsilon}) + \lambda \bar{C} (\log(\frac{1}{\epsilon}))^2)$

Table 1: Comparison with representative quantum algorithms for implementing $A = \sum_{j=0}^{J-1} c_j U_j$. We assume $c_j > 0$, U_j are constructed from a shared set of primitive oracles and oracle-independent gates, and each U_j has query complexity C_j . The parameter $a \geq \|A\|$ is a known upper bound of $\|A\|$, $\lambda = \sum c_j$ is the coefficient 1-norm (we assume $a \leq \lambda$), $C_{\max} = \max_j C_j$ denotes the maximum complexity, and $\bar{C} = \sum c_j C_j / \lambda$ denotes the weighted average cost. The “query complexity per run” is the number of oracle calls to construct a block-encoding of A , “rescaling” is the rescaling factor of this block-encoding, and “overall query complexity” refers to the product of the former two columns, which is a representative complexity estimate when applying LCU on an initial state with standard amplitude amplification.

Hamiltonian simulation. Among existing Hamiltonian simulation algorithms, quantum signal processing, qubitization and QSVT achieve optimal query dependence $\mathcal{O}(\alpha T + \log(1/\epsilon))$ for a given block-encoding with rescaling factor α [14, 28, 29]. With the standard sparse block-encoding [14], the rescaling factor is $\mathcal{O}(d)$, and thus the query complexity to the sparse oracles is $\mathcal{O}(dT + \log(1/\epsilon))$, which scales linearly in the sparsity d . Compared to these approaches, our algorithm almost quadratically improves the sparsity dependence, at the expense of extra poly-logarithmic factors in T and $1/\epsilon$.

For a sparse Hamiltonian simulation, ref. [16] proposes an algorithm through a decomposition by entry magnitude and recursive interaction-picture simulation, obtaining $\mathcal{O}(\sqrt{d} T (\sqrt{d} T / \epsilon)^{o(1)})$ queries. Here the $o(1)$ power represents extra sub-polynomial but super-logarithmic factors. Our sparse Hamiltonian simulation builds on this decomposition, and further achieves a better complexity by reducing the extra $(\sqrt{d} T)^{o(1)}$ to $\text{polylog}(dT)$ and improving the accuracy dependence from super-logarithmic to poly-logarithmic. We would like to remark that this comparison concerns the case when the matrix is normalized according to its spectral norm. In a more general case without such a normalization, [16] appears to have a better norm dependence (i.e., the matrix 1-to-2 norm) than ours (suggested by Theorem 5.3).

We summarize this complexity comparison in Table 2.

Quantum linear systems. Starting with the HHL algorithm [30], there have been substantial works on quantum linear system algorithms with improved condition number and precision dependence (see, e.g., [1, 27, 31–34]). Among them, the discrete adiabatic algorithm [27] and the kernel reflection approach [34] achieve $\mathcal{O}(\alpha \kappa \log(1/\epsilon))$ block-encoding queries for $\|A\| = 1$ and normalization α . Combining these methods with a standard sparse block-encoding gives a worst-case dependence linear in d . Our algorithm keeps the main track of the algorithm

Method	Query complexity
QSP/Qubitization/QSVT [14, 28, 29]	$\mathcal{O}(dT + \log(1/\epsilon))$
Recursive interaction-picture simulation [16]	$\mathcal{O}((\sqrt{d}T)^{1+o(1)}/\epsilon^{o(1)})$
Our work (Theorem 5.5)	$\mathcal{O}(\sqrt{d}T \text{polylog}(dT/\epsilon))$

Table 2: Comparison with representative quantum algorithms for sparse Hamiltonian simulation e^{-iAT} . Here A is a Hamiltonian with $\|A\| = 1$ and sparsity d , and we assume access to its standard sparse oracle. The parameter ϵ denotes the target operator norm error.

unchanged, but only alters the construction of the block-encoding input model. This again can almost quadratically improve the sparsity dependence at the expense of extra poly-logarithmic factors in κ and $1/\epsilon$.

Alternatively, the work [19] separately improves query complexity to the right-hand-side state using tunable VTAA. Combining their solver with our block-encoding preserves this benefit while reducing the number of queries to the sparse matrix access. Recent algorithms also refine the dependence on the spectral properties of a particular instance beyond its worst-case condition number [35, 36], and our results address the complementary dependence on sparsity.

Tailored for sparse linear system solvers, the work [16] also already obtained square-root sparsity dependence with subpolynomial overhead, namely $\mathcal{O}(\sqrt{d}\kappa(\sqrt{d}\kappa/\epsilon)^{o(1)})$. Our algorithm achieves $\mathcal{O}(\sqrt{d}\kappa \text{polylog}(\kappa d/\epsilon))$, thus reducing the $(\sqrt{d}\kappa/\epsilon)^{o(1)}$ overhead to $\text{polylog}(d\kappa/\epsilon)$.

We summarize these comparisons in Table 3.

Method	Queries to the matrix oracles	Queries to the state preparation
Discrete adiabatic [27]/ Kernel reflection [34]	$\mathcal{O}(d\kappa \log(1/\epsilon))$	$\mathcal{O}(d\kappa \log(1/\epsilon))$
Tunable VTAA [19]	$\mathcal{O}(d\kappa \log \kappa \log(\log \kappa/\epsilon))$	$\mathcal{O}(\kappa/\ A^{-1} b\rangle\)$
Recursive interaction-picture simulation [16]	$\mathcal{O}((\sqrt{d}\kappa)^{1+o(1)}/\epsilon^{o(1)})$	$\mathcal{O}(\kappa/\ A^{-1} b\rangle\)$
Our work (Theorem 5.6)	$\mathcal{O}(\sqrt{d}\kappa \text{polylog}(d\kappa/\epsilon))$	$\mathcal{O}(\kappa/\ A^{-1} b\rangle\)$

Table 3: Comparison with representative quantum algorithms for solving linear system $Ax = b$. Here A is a Hermitian matrix with $\|A\| = 1$, sparsity d , and condition number κ . The right hand side b is a normalized vector. We assume access to the standard sparse oracle of A , and the state preparation oracle of $|b\rangle$. The numbers of queries to these different oracles are given separately in the latter two columns. The parameter ϵ denotes the target error in the normalized state measured by the vector 2-norm.

Transducers. The main technical tool of our algorithm is the transducer framework introduced in [21], as a framework for quantum state conversion and composition, with costs that reflect how subroutines are used. Very recently, several transducer-based quantum algorithms have been proposed, including time-dependent Hamiltonian simulation [23, 24], Lindblad simulation [25], quantum walks [22] and ground state energy estimation [37]. Our work further extends the applicable regime of the transducer framework to the fundamental LCU task. We also establish two technical tools in the transducer framework with improved performance, which might be of independent interest.

First, we develop a general high-order method that converts a transducer back into a standard block-encoding of its public action. The general implementation in [21] uses $\mathcal{O}(W/\epsilon^2)$ calls to the transducer to achieve state error ϵ , where W bounds the squared catalyst norm (which can be roughly understood as the weighted cost). In contrast, if the private block T satisfies

that $I - T$ is invertible, and a bound $K \geq \|(I - T)^{-1}\|$ on the resolvent is known, our construction establishes an $\mathcal{O}(K \log(1/\epsilon))$ complexity. This achieves an exponential improvement in precision, but the norm dependence is changed from W to K , the norm of the resolvent and might be worse than the catalyst norm in the worst case. For our LCU construction, however, the resolvent bound retains the weighted cost and enables logarithmic precision dependence. Notice that such an exponential improvement has been previously achieved in [23], while relying more delicately on the specific structure of the matrices.

To bound the resolvent norm K throughout the algorithm, we develop a more detailed analysis on the transducer of unitary and/or transducer composition in Section 3.1. This extends the result in [21] with a complementary operator-level theory tailored to block-encodings and a rigorous norm bound on the resolvent.

Besides technical tools, we also describe a simple Schur-complement interpretation of transducers. Roughly speaking, a transducer embeds the desired transformation into a larger unitary system, from which the effective operation is recovered by eliminating auxiliary variables. This viewpoint connects the transducer framework to standard Gaussian block elimination in linear algebra and provides useful intuition for both its composition rules and our complexity analysis.

Concurrent work. Bravo-Prieto, Harrow, and Kothari [38] obtain the optimal sparse-matrix query complexity $\Theta(\kappa\sqrt{d}\log(1/\epsilon))$ for quantum linear systems problem. They also construct a block-encoding of any d -sparse Hermitian matrix with spectral norm at most one, using $\mathcal{O}(\sqrt{d}\log(1/\epsilon))$ queries. Their method directly block-encodes an enlarged linear system whose inverse contains the desired operator or solution, further removing extra poly-logarithmic factors compared to our Result 1.2 and Result 1.4. Our main result addresses a different task of linear combinations of matrices, which applies beyond sparse-matrix access and provides a weighted-cost LCU primitive, together with resolvent bounds and high-precision transducer conversion.

1.4 Organization

The remainder of this paper is organized as follows. Section 2 introduces the problem setting and reviews the basic tools of quantum linear algebra and transducers, together with a Schur-complement interpretation of the transducer framework. Section 3 develops the transducer toolkit used throughout the paper, including composition rules with resolvent-norm bounds and a high-order finite-reuse procedure for converting a transducer into a block-encoding of its action. Section 4 presents the transducer-based LCU algorithm and its complexity estimate. Section 5 applies the framework to sparse matrices, deriving an improved sparse block-encoding and its applications to sparse Hamiltonian simulation and quantum linear systems. Section 6 concludes the work with potential future directions.

2 Preliminaries

In this section, we define the goal and conditions of the linear combination of matrices task considered in this work. We also review some basic concepts and results in quantum linear algebra and transducers, which will be the building blocks of our algorithm.

2.1 Problem setting

Consider a linear combination of matrices in the form of

$$A = \sum_{j=0}^{J-1} c_j A_j. \quad (2)$$

Here A_j 's are general matrices, and c_j 's are positive real coefficients². The main task of this work is to construct a block-encoding of A .

We assume access to the block-encoding U_j of A_j/α_j with normalization factor $\alpha_j \geq \|A_j\|$. Additionally, we make an important assumption that U_j are related in the following sense.

Assumption 2.1 (Shared primitive access). *The circuits of U_j have known descriptions with C_j calls to a fixed finite family of primitive oracles. Specifically, each U_j has decomposition $U_j = G_j^{[C_j]} Q_j^{[C_j]} \dots G_j^{[1]} Q_j^{[1]} G_j^{[0]}$, where $G_j^{[k]}$'s are known oracle-free unitaries, $Q_j^{[k]}$ belong to a fixed finite primitive oracle family and its adjoints, and assume that the coherent operator $\sum_{j,k} |j, k\rangle \langle j, k| \otimes Q_j^{[k]}$ can be implemented with $\mathcal{O}(1)$ primitive queries.*

Throughout this work, the query complexity is counted as the number of queries to the primitive oracles in Assumption 2.1. This assumption allows the circuits U_j to have different lengths and oracle-free gates while sharing a fixed family of primitive oracles. It requires their next primitive call to be selected coherently from the labels using $\mathcal{O}(1)$ queries.

Assumption 2.1 is natural in many structured settings. For example, the quantum phase estimation circuit consists of the controlled U^j applications for the unitary U , and the circuits $U_j = U^j$ satisfy our shared primitive oracle assumption. Another example is multiple QSVT circuits based on a common block-encoding U_H , which may have different polynomial degrees and phase sequences, but their oracle calls use only U_H and $U_H^\dagger$ [14]. With controlled access to these primitives, they satisfy the assumption.

This setting also includes the main applications considered in this work. In both sparse Hamiltonian simulation and sparse linear-system solving, the relevant block-encodings are ultimately constructed from the same sparse-access oracles of the input matrix. Thus the family of circuits appearing in these applications naturally shares a common primitive-query interface and satisfies Assumption 2.1. The assumption therefore does not require the U_j 's themselves to have the same circuit structure; it only requires their oracle-dependent components to arise from a common coherently accessible set of primitive oracles.

For the coefficients, we assume that they are known in the sense that preparing a quantum state encoding the information of $\sqrt{c_j}$ is efficient. For arbitrary unstructured coefficients, such a state can be prepared using $\mathcal{O}(J)$ elementary gates by standard generic state-preparation procedures [39], while substantially more efficient implementations are possible when the coefficients possess additional structure [40]. Nevertheless, they are independent of the primitive oracles for U_j , so the usage of the coefficients will not be counted in the query complexity.

In the remaining part of this work, we will only consider the special case where both A_j and U_j are Hermitian. This additional assumption does not sacrifice the generality due to the Hermitianization technique [32]. Specifically, append one dilation qubit and let

$$\tilde{A}_j = |0\rangle \langle 1| \otimes A_j + |1\rangle \langle 0| \otimes A_j^\dagger = \begin{pmatrix} 0 & A_j \\ A_j^\dagger & 0 \end{pmatrix} \quad (3)$$

and

$$\tilde{U}_j = |0\rangle \langle 1| \otimes U_j + |1\rangle \langle 0| \otimes U_j^\dagger. \quad (4)$$

Then $\tilde{A}_j$ is Hermitian, and $\tilde{U}_j$ is a Hermitian unitary. Furthermore, we have $(I \otimes \langle 0| \otimes I) \tilde{U}_j (I \otimes |0\rangle \otimes I) = \tilde{A}_j/\alpha_j$, i.e., $\tilde{U}_j$ is a Hermitian block-encoding of $\tilde{A}_j/\alpha_j$ after reordering the sub-blocks. Therefore, once we know how to implement the linear combination of Hermitian matrices via their Hermitian block-encodings, we can directly obtain a block-encoding $\tilde{U}$ of $\tilde{A}/\tilde{\alpha}$ with

$$\tilde{A} = \sum_{j=0}^{J-1} c_j \tilde{A}_j = \begin{pmatrix} 0 & A \\ A^\dagger & 0 \end{pmatrix}. \quad (5)$$

²The positive real assumption does not lose generality, as the phase factor of any potential complex coefficients can be absorbed in the matrix A_j .

Then $\tilde{U}(X \otimes I)$ (and regarding the dilation qubit as the ancilla qubit as well) gives the desired block-encoding of A .

2.2 Quantum linear algebra primitives

Block-encodings provide a standard way to represent a possibly non-unitary operator as a block of a larger unitary, enabling one to apply quantum algorithms, such as QSP and QSVT, to general linear operators. The notion was implicit in earlier Hamiltonian simulation algorithms [29] and was later formalized in [14, 32]. We use the following standard definition.

Definition 2.2 (Block-encoding). *Let A be an s -qubit operator, $\alpha, \epsilon \in \mathbb{R}_{\geq 0}$ and $a \in \mathbb{N}$. An $(s+a)$ -qubit unitary U_A is called an (α, a, ϵ) -block-encoding of A , if*

$$\|A - \alpha \cdot (\langle 0^a | \otimes I) U_A (|0^a\rangle \otimes I)\| \leq \epsilon. \quad (6)$$

Intuitively,

$$U_A = \begin{pmatrix} A/\alpha & * \\ * & * \end{pmatrix}. \quad (7)$$

For a unitary decomposition $A = \sum_{j=0}^{J-1} c_j U_j$, the standard LCU [2, 15] framework yields a block-encoding of A as follows.

Lemma 2.3 (Standard LCU). *Let $A = \sum_{j=0}^{J-1} c_j U_j$, where each U_j is unitary, $c_j > 0$ and $c = \sum_j c_j$. Let V be a unitary such that*

$$V |0\rangle = \frac{1}{\sqrt{c}} \sum_{j=0}^{J-1} \sqrt{c_j} |j\rangle. \quad (8)$$

Then the quantum circuit

$$(V^\dagger \otimes I) \left(\sum_{j=0}^{J-1} |j\rangle \langle j| \otimes U_j \right) (V \otimes I) \quad (9)$$

is an $(c, \lceil \log J \rceil, 0)$ -block-encoding of A . It uses one application of $V, V^\dagger$, and one controlled application of each U_j .

Given a block-encoding of a Hermitian matrix, QSVT allows one to implement polynomial transformations of it.

Lemma 2.4 (QSVT, Theorem 56 of [14]). *Suppose that U is an (α, a, ϵ) -block-encoding of a Hermitian matrix A . Let $P \in \mathbb{R}[x]$ be a degree- d polynomial satisfying $|P(x)| \leq 1/2$ for all $x \in [-1, 1]$. Then there is a quantum circuit $\tilde{U}$, which is a $(1, a + 2, 4d\sqrt{\epsilon/\alpha})$ -block-encoding of $P(A/\alpha)$, and consists of d applications of U and $U^\dagger$, a single application of controlled- U and $\mathcal{O}((a+1)d)$ other one- and two-qubit gates.*

2.3 Transducer

A transducer represents a unitary transformation through a unitary S acting on a direct sum of public and private spaces. It transforms the public input as desired while returning an input-dependent private vector, called the catalyst, unchanged. Here we give a formal definition of the transducer framework of [21].

Definition 2.5 (Transducer). *Let $\mathcal{H}_{\text{pub}}$ and $\mathcal{H}_{\text{priv}}$ be two finite-dimensional Hilbert spaces. A unitary $S : \mathcal{H}_{\text{pub}} \oplus \mathcal{H}_{\text{priv}} \rightarrow \mathcal{H}_{\text{pub}} \oplus \mathcal{H}_{\text{priv}}$ is called a transducer of $V : \mathcal{H}_{\text{pub}} \rightarrow \mathcal{H}_{\text{pub}}$, if there exists a linear mapping $\Gamma : \mathcal{H}_{\text{pub}} \rightarrow \mathcal{H}_{\text{priv}}$ such that*

$$S(\psi \oplus \Gamma\psi) = V\psi \oplus \Gamma\psi, \quad \forall \psi \in \mathcal{H}_{\text{pub}}. \quad (10)$$

Here $\mathcal{H}_{\text{pub}}$ and $\mathcal{H}_{\text{priv}}$ are called the public and private spaces, respectively. We call V the *public action*, $\Gamma\psi$ the *catalyst*, and Γ the *catalyst map*. The catalyst vector is returned unchanged by S . For a fixed choice of catalyst map Γ , we define the input-dependent transduction complexity and its worst-case value by

$$W(S, \psi) := \|\Gamma\psi\|^2, \quad w(S) := \sup_{\|\psi\|=1} W(S, \psi) = \|\Gamma\|^2. \quad (11)$$

When the catalyst map is not unique, these quantities refer to the specified choice of Γ . The catalyst serves as a mathematical witness and need not be prepared. The following result approximately implements the public action using repeated applications of the transducer S , without access to Γ .

Lemma 2.6 (Transducer implementation, Theorem 5.5 of [21]). *Let S be a transducer with public action V and catalyst map Γ . For every integer $N \geq 1$ and initial state $|\psi\rangle \in \mathcal{H}_{\text{pub}}$, there is a quantum circuit that outputs $|\tau\rangle$ such that*

$$\| |\tau\rangle - V|\psi\rangle \| \leq 2\sqrt{\frac{W(S, \psi)}{N}}. \quad (12)$$

The circuit uses N controlled applications of S and $\mathcal{O}(N)$ other elementary operations.

From the above result, implementing V to additive error ϵ requires $\mathcal{O}(W/\epsilon^2)$ calls to S . Later, in Theorem 3.5, we give a different implementation whose precision dependence is only logarithmic when the private-block resolvent is suitably bounded.

Some basic properties of the transducer directly follow from its definition.

Lemma 2.7 (Basic properties of a transducer). *Let S be a transducer of V with catalyst map Γ , and its matrix representation under the standard basis is*

$$S = \begin{pmatrix} D & E \\ B & T \end{pmatrix}. \quad (13)$$

Suppose that $I - T$ is invertible, and define

$$K(S) = \|(I - T)^{-1}\|. \quad (14)$$

Then:

1. *The action V is unitary, and has the representation $V = D + E(I - T)^{-1}B$.*
2. *The catalyst has the representation $\Gamma = (I - T)^{-1}B$*
3. *We have the following identities: $E(I - T)^{-1} = V\Gamma^\dagger$, and $\Gamma\Gamma^\dagger = (I - T)^{-1} + (I - T^\dagger)^{-1} - I$.*
4. *$w(S) \leq 2K(S) - 1$.*

Proof. The proof follows from elementary block-matrix manipulations together with the unitarity of S . By definition, for every $\psi \in \mathcal{H}_{\text{pub}}$ we have

$$S \begin{pmatrix} \psi \\ \Gamma\psi \end{pmatrix} = \begin{pmatrix} V\psi \\ \Gamma\psi \end{pmatrix}. \quad (15)$$

Since S is unitary, $\|\psi\|^2 + \|\Gamma\psi\|^2 = \|V\psi\|^2 + \|\Gamma\psi\|^2$, and hence $\|V\psi\| = \|\psi\|$. Thus V is unitary. Writing the defining relation blockwise gives

$$D + E\Gamma = V, \quad B + T\Gamma = \Gamma. \quad (16)$$

The second identity gives $\Gamma = (I - T)^{-1}B$. Substituting this into the first identity yields $V = D + E(I - T)^{-1}B$. Since S is unitary,

$$S^\dagger \begin{pmatrix} V\psi \\ \Gamma\psi \end{pmatrix} = \begin{pmatrix} \psi \\ \Gamma\psi \end{pmatrix}. \quad (17)$$

The second block equation gives $E^\dagger V + T^\dagger \Gamma = \Gamma$, and so $E(I - T)^{-1} = V\Gamma^\dagger$.

Let $R := (I - T)^{-1}$. Since $\Gamma = RB$ and S is unitary, the lower-right block of $SS^\dagger = I$ gives $BB^\dagger + TT^\dagger = I$. Hence $\Gamma\Gamma^\dagger = RBB^\dagger R^\dagger = R(I - TT^\dagger)R^\dagger$. Using $I - TT^\dagger = (I - T) + (I - T^\dagger) - (I - T)(I - T^\dagger)$, we obtain $\Gamma\Gamma^\dagger = (I - T)^{-1} + (I - T^\dagger)^{-1} - I$. Finally, since $\Gamma\Gamma^\dagger$ is positive semidefinite,

$$w(S) = \max_{\|x\|=1} \langle x, \Gamma\Gamma^\dagger x \rangle = \max_{\|x\|=1} (2\operatorname{Re}\langle x, Rx \rangle - 1) \leq 2\|R\| - 1 = 2K(S) - 1. \quad (18)$$

This proves the claimed bound. $\square$

2.4 Schur-complement interpretation of transducers

The algebraic structure underlying a transducer is essentially a Schur-complement reduction. Schur complements form a classical and extensively studied subject in matrix analysis, with applications ranging from block Gaussian elimination and matrix inversion to numerical linear algebra, optimization, probability, control theory, and effective operator constructions [41]. From this viewpoint, the transducer action can be understood through the effective operator obtained by eliminating the private variables. This perspective provides a unified interpretation of many of the basic identities and composition rules used below.

Let

$$S = \begin{pmatrix} D & E \\ B & T \end{pmatrix} \quad (19)$$

be a transducer and assume that $I - T$ is invertible. The defining relation

$$S \begin{pmatrix} x \\ \Gamma x \end{pmatrix} = \begin{pmatrix} Vx \\ \Gamma x \end{pmatrix} \quad (20)$$

admits a natural interpretation in terms of block elimination. Indeed,

$$(I - S) \begin{pmatrix} x \\ \Gamma x \end{pmatrix} = \begin{pmatrix} (I - V)x \\ 0 \end{pmatrix}. \quad (21)$$

This means that eliminating the private component amounts to taking the Schur complement of the lower-right block $I - T$ in $I - S$. Explicitly,

$$\operatorname{Schur}_{I-T}(I - S) = (I - D) - (-E)(I - T)^{-1}(-B) = I - V. \quad (22)$$

In other words, $I - V$ is precisely the reduced operator obtained by eliminating the private variables from $I - S$, while V is the corresponding action of the transducer on the public space.

Concretely, for a given public vector x , the second block equation uniquely determines the private vector that must accompany it, $y = (I - T)^{-1}Bx = \Gamma x$. Substituting this solution into the first block equation then yields the reduced action on the public space. This is precisely the usual interpretation of the Schur complement: one first solves for the variables to be eliminated and then substitutes them back to obtain the effective equation for the variables that are retained.

The preceding relation can equivalently be written as the block elimination identity

$$\begin{pmatrix} I & E(I - T)^{-1} \\ 0 & I \end{pmatrix} (I - S) = \begin{pmatrix} I - V & 0 \\ -B & I - T \end{pmatrix}. \quad (23)$$

Thus, by eliminating the private variables, $I - S$ is reduced in a single block Gaussian elimination step to a block-triangular matrix whose diagonal blocks are $I - V$ and $I - T$. In this sense, the reduced operator $I - V$, and hence the action V of the transducer on the public space, arises directly from block Gaussian elimination of the private variables.

This also clarifies the roles of $K(S)$ and the catalyst map Γ . The matrix $I - T$ serves as the pivot in the block elimination, with $K(S) = \|(I - T)^{-1}\|$ measuring the worst-case amplification in solving $(I - T)y = b$. By contrast, $\Gamma = (I - T)^{-1}B$ restricts this inverse to the particular right-hand sides Bx generated by the public variables. Equivalently, Γ is the back-substitution map that reconstructs the corresponding private component, $y = \Gamma x$.

The Schur determinant formula also gives

$$\det(I - S) = \det(I - T) \det(I - V). \quad (24)$$

Since $I - T$ is assumed to be invertible, $\det(I - S) = 0$ if and only if $\det(I - V) = 0$. Equivalently, from (21) the eigenspaces of S and V corresponding to the eigenvalue 1 are naturally isomorphic via

$$x \mapsto \begin{pmatrix} x \\ \Gamma x \end{pmatrix}. \quad (25)$$

So $\dim \ker(I - S) = \dim \ker(I - V)$. This is also a standard feature of Schur-complement reduction: eliminating part of the variables retains the spectral information relevant to the remaining variables. In the present setting, since the reduction is applied to $I - S$, it preserves precisely the spectral information at eigenvalue 1.

A particularly useful property of Schur complements is the quotient formula, which states that, whenever the relevant Schur complements are well defined, successive elimination of nested blocks agrees with eliminating those blocks at once [41, 42]. This gives a natural interpretation of the transducer composition in Lemma 3.2.

Suppose

$$S_j = \begin{pmatrix} D_j & E_j \\ B_j & T_j \end{pmatrix} \quad (26)$$

is a transducer with action $V_j = D_j + E_j(I - T_j)^{-1}B_j$. In the composite construction, let q_j denote the state entering the public space of the j -th transducer, and let p_j denote its local private variable. They satisfy

$$p_j = B_j q_j + T_j p_j, \quad q_{j+1} = D_j q_j + E_j p_j. \quad (27)$$

Eliminating the local private variable p_j gives

$$p_j = (I - T_j)^{-1} B_j q_j = \Gamma_j q_j, \quad (28)$$

and therefore

$$q_{j+1} = [D_j + E_j(I - T_j)^{-1} B_j] q_j = V_j q_j. \quad (29)$$

Thus, after eliminating all local private variables, the composite transducer reduces to the clock propagation $q_{j+1} = V_j q_j$, or equivalently,

$$q_1 = V_0 q_0, \quad q_2 = V_1 V_0 q_0, \quad \dots, \quad q_L = V_{L-1} \cdots V_0 q_0. \quad (30)$$

One may then eliminate the intermediate clock variables $q_1, \dots, q_{L-1}$, which yields the overall action $V_{L-1} \cdots V_1 V_0$. Equivalently, one may eliminate all local private variables and intermediate clock variables in a single Schur reduction. The quotient property of Schur complements guarantees that the two procedures give the same result.

In summary, the Schur-complement viewpoint gives the following dictionary:

Schur-complement language	Transducer language
variables retained after elimination	public space
variables eliminated	private space
reduced operator	$I - V$
back substitution	$\Gamma = (I - T)^{-1}B$
inverse of the elimination pivot	$(I - T)^{-1}$
worst-case amplification in elimination	$K(S) = \ (I - T)^{-1}\ $
successive elimination	transducer composition

Therefore, at the algebraic level, a transducer may be viewed as a unitary realization of a Schur-complement reduction. The algorithmic content of the transducer framework lies not in the Schur-complement identity itself, but in constructing a larger unitary S that is inexpensive to implement while the induced action V on the public space may be substantially more expensive to realize directly. Equivalently, the Schur complement of $I - S$ with respect to the private block is $I - V$. The clock constructions in Lemmas 3.1 and 3.2 illustrate this principle: they encode a sequential computation into the enlarged private space, so that eliminating the private variables recovers the desired composition on the public space.

3 Transducer toolkit

In this section, we discuss some advanced transducer's propositions, which can be roughly categorized into two classes. The first part is the construction of a single transducer with action being the sequential composition of multiple unitaries, providing a methodology of transiting the query complexity brought by the sequential operations to the norms related to the private space of the transducer. The second part discusses how to turn a transducer to a block-encoding of its action without the need of explicit catalyst, formally called catalyst removal via finite reuse.

3.1 Transducers for composition

Lemma 3.1 (Transducer for unitary composition). *Let $V = G_L O_{L-1} G_{L-1} \cdots G_1 O_0 G_0$ be a unitary circuit where O_k represent one oracle call and G_k are oracle-independent unitaries. Then there exists a transducer S with action V such that:*

1. The transducer S is in the form of

$$S = \begin{pmatrix} 0 & E \\ B & T \end{pmatrix} \quad (31)$$

with $T^L = 0$.

2. Each implementation of S requires one call to the select oracle $\sum_{k=0}^{L-1} |k\rangle \langle k| \otimes O_k$.
3. We have the following identities and estimates:

$$\begin{aligned} V &= E(I - T)^{-1}B, & K(S) &= \|(I - T)^{-1}\| \leq L, \\ \|(I - T)^{-1}B\psi\|^2 &= L\|\psi\|^2, & \|E(I - T)^{-1}\| &= \sqrt{L}. \end{aligned} \quad (32)$$

Proof. Denote $O_{-1} = I$ and $V_k = G_k O_{k-1} \cdots G_1 O_0 G_0$ (intuitively V_k is a part of the circuit of

V truncated at k -th oracle calls, with $V_0 = G_0$ and $V_L = V$). Let

$$S = \sum_{k=0}^{L-1} |k+1\rangle \langle k| \otimes G_k O_{k-1} + |0\rangle \langle L| \otimes G_L O_{L-1}$$

$$= \begin{pmatrix} 0 & & & & & G_L O_{L-1} \\ G_0 & 0 & & & & \\ & G_1 O_0 & 0 & & & \\ & & G_2 O_1 & \ddots & & \\ & & & \ddots & \ddots & \\ & & & & G_{L-1} O_{L-2} & 0 \end{pmatrix}. \quad (33)$$

Then the unitary S given in Eq. (33) is a transducer of V , because for any vector ψ ,

$$S(\psi \oplus (V_0\psi \oplus V_1\psi \oplus \cdots \oplus V_{L-1}\psi)) \quad (34)$$

$$= S \left(|0\rangle \otimes \psi + \sum_{k=1}^L |k\rangle \otimes V_{k-1}\psi \right) \quad (35)$$

$$= |1\rangle \otimes G_0\psi + \sum_{k=1}^{L-1} |k+1\rangle \otimes G_k O_{k-1} V_{k-1}\psi + |0\rangle \otimes G_L O_{L-1} V_{L-1}\psi \quad (36)$$

$$= |1\rangle \otimes V_0\psi + \sum_{k=2}^L |k\rangle \otimes V_{k-1}\psi + |0\rangle \otimes V_L\psi \quad (37)$$

$$= V\psi \oplus (V_0\psi \oplus V_1\psi \oplus \cdots \oplus V_{L-1}\psi). \quad (38)$$

The corresponding catalyst map is

$$\Gamma = \begin{pmatrix} V_0 \\ V_1 \\ \vdots \\ V_{L-1} \end{pmatrix}, \quad (39)$$

and $E = [0, 0, \dots, 0, G_L O_{L-1}]$, $B = [G_0; 0; \dots; 0]$, and

$$T = \begin{pmatrix} 0 & & & & \\ G_1 O_0 & 0 & & & \\ & G_2 O_1 & \ddots & & \\ & & \ddots & \ddots & \\ & & & G_{L-1} O_{L-2} & 0 \end{pmatrix}. \quad (40)$$

It is clear that $T^L = 0$ since T is a lower triangular L -by- L block matrix with all the diagonal blocks to be 0. Furthermore, notice that S can be rewritten as

$$S = \left(\left(\sum_{k=0}^{L-1} |k+1\rangle \langle k| + |0\rangle \langle L| \right) \otimes I \right) \left(\sum_{k=0}^L |k\rangle \langle k| \otimes G_k \right) \left(I \oplus \sum_{k=0}^{L-1} |k\rangle \langle k| \otimes O_k \right), \quad (41)$$

so its implementation requires one query to the select oracle of O_j 's.

Now let us prove the identities and estimates claimed in the lemma. The identity $V = E(I - T)^{-1}B$ directly follows from Lemma 2.7 and noticing that the left upper block of S here is 0. Since $T^L = 0$, we have

$$(I - T)^{-1} = I + T + T^2 + \cdots + T^{L-1}, \quad (42)$$

and thus $K(S) = \|(I - T)^{-1}\| \leq L$. A direct computation shows that³

$$(I - T)^{-1}B\psi = [V_0\psi; V_1\psi; \dots; V_{L-1}\psi], \quad (43)$$

so $\|(I - T)^{-1}B\psi\|^2 = \sum_{k=0}^{L-1} \|V_k\psi\|^2 = L\|\psi\|^2$. For the last equation, by $(I - T)^{-1} = I + T + T^2 + \dots + T^{L-1}$, we can compute that

$$E(I - T)^{-1} = (R_0 \ R_1 \ \dots \ R_{L-2} \ R_{L-1}), \quad (44)$$

where $R_k = G_L O_{L-1} G_{L-1} O_{L-2} \dots G_{k+1} O_k$ for $0 \leq k \leq L-1$. In particular, since each R_k is unitary, $E(I - T)^{-1}(E(I - T)^{-1})^\dagger = \sum_{k=0}^{L-1} R_k R_k^\dagger = LI$, and so $\|E(I - T)^{-1}\| = \sqrt{L}$. $\square$

In some scenarios, the intermediate unitaries in a circuit are realized through their transducers, so we also need a result for the composition of multiple transducers. We first discuss a simple case where all the unitaries are given by their transducers. Our construction is closely related to the parallel implementation of sequential transducers in [21, Proposition 9.10 and Theorem 10.5]. There, the intermediate states of a sequential composition are stored in the catalyst, which leads to the same additive bound on the catalyst norm as Eq. (45). Our result gives an explicit block-matrix realization of this construction and, in addition, derives the resolvent bound in Eq. (46), which will be essential for the finite-reuse implementation below.

Lemma 3.2 (Transducer for pure transducer composition). *For $0 \leq j \leq L-1$, let V_j be unitaries of the same dimension, and $S_j = \begin{pmatrix} D_j & E_j \\ B_j & T_j \end{pmatrix}$ be the transducer of V_j with catalyst map Γ_j . Then there exists a transducer S of $V = V_{L-1} \dots V_1 V_0$ such that*

$$w(S) \leq L - 1 + \sum_{j=0}^{L-1} w(S_j), \quad (45)$$

$$K(S) \leq \max_j K(S_j) + (L - 1)(1 + \max_j w(S_j)), \quad (46)$$

using one query to the select oracle of S_j .

Proof. We construct S as⁴

$$S = \begin{pmatrix} 0 & 0 & \dots & 0 & D_{L-1} & 0 & 0 & \dots & 0 & E_{L-1} \\ D_0 & 0 & \dots & 0 & 0 & E_0 & 0 & \dots & 0 & 0 \\ 0 & D_1 & \dots & 0 & 0 & 0 & E_1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & D_{L-2} & 0 & 0 & 0 & \dots & E_{L-2} & 0 \\ B_0 & 0 & 0 & \dots & 0 & T_0 & 0 & 0 & \dots & 0 \\ 0 & B_1 & 0 & \dots & 0 & 0 & T_1 & 0 & \dots & 0 \\ 0 & 0 & B_2 & \dots & 0 & 0 & 0 & T_2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & B_{L-1} & 0 & 0 & 0 & \dots & T_{L-1} \end{pmatrix} \quad (47)$$

Notice that S can be constructed through one call to the select oracle of S_j and permutations of the block order.

Now we show that S is the desired transducer with the public space being the upper left block. Let ψ_0 be an arbitrary vector of dimension the same as the column vector of V , and choose

$$\psi_j = V_{j-1} V_{j-2} \dots V_0 \psi_0, \quad j \geq 1. \quad (48)$$

³Here and below, $[A_1; \dots; A_k]$ denotes the block column obtained by vertically stacking the vectors or matrices $A_1, \dots, A_k$.

⁴See Appendix B for an explanation of this construction.

Denote

$$u = [\psi_0; \psi_1; \cdots; \psi_{L-1}; \Gamma_0 \psi_0; \Gamma_1 \psi_1; \cdots; \Gamma_{L-1} \psi_{L-1}] = \psi_0 \oplus \Gamma \psi_0, \quad (49)$$

where

$$\Gamma = [V_0; V_1 V_0; \cdots; V_{L-2} \cdots V_1 V_0; \Gamma_0; \Gamma_1 V_0; \Gamma_2 V_1 V_0; \cdots; \Gamma_{L-1} V_{L-2} \cdots V_1 V_0]. \quad (50)$$

Then $Su = [x_0; x_1; \cdots; x_{L-1}; y_0; y_1; \cdots; y_{L-1}]$, where

$$x_0 = D_{L-1} \psi_{L-1} + E_{L-1} \Gamma_{L-1} \psi_{L-1}, \quad x_{j+1} = D_j \psi_j + E_j \Gamma_j \psi_j, \quad 0 \leq j \leq L-2, \quad (51)$$

$$y_j = B_j \psi_j + T_j \Gamma_j \psi_j, \quad 0 \leq j \leq L-1. \quad (52)$$

According to Lemma 2.7, we have $D_j + E_j \Gamma_j = V_j$ and $B_j + T_j \Gamma_j = \Gamma_j$, so

$$x_0 = V_{L-1} \psi_{L-1}, \quad x_{j+1} = V_j \psi_j, \quad 0 \leq j \leq L-2, \quad (53)$$

$$y_j = \Gamma_j \psi_j, \quad 0 \leq j \leq L-1. \quad (54)$$

From the iterative relation of x_j , we have $x_0 = V_{L-1} \cdots V_1 V_0 \psi_0 = V \psi_0$ and $x_j = V_{j-1} \cdots V_1 V_0 \psi_0$. So we can write

$$Su = S(\psi_0 \oplus \Gamma \psi_0) \quad (55)$$

$$= [V \psi_0; V_0 \psi_0; V_1 V_0 \psi_0; \cdots; V_{L-2} \cdots V_1 V_0 \psi_0; \Gamma_0 \psi_0; \Gamma_1 V_0 \psi_0; \Gamma_2 V_1 V_0 \psi_0; \cdots; \Gamma_{L-1} V_{L-2} \cdots V_1 V_0 \psi_0] \quad (56)$$

$$= V \psi_0 \oplus \Gamma \psi_0, \quad (57)$$

which verifies that S defined in Eq. (47) is a transducer of V with catalyst Γ defined in Eq. (50).

Now let us bound the relevant norms of S . From the definition of Γ in Eq. (50), for any vector ψ , we have

$$\|\Gamma \psi\|^2 = \|V_0 \psi\|^2 + \|V_1 V_0 \psi\|^2 + \cdots + \|V_{L-2} \cdots V_1 V_0 \psi\|^2 \quad (58)$$

$$+ \|\Gamma_0 \psi\|^2 + \|\Gamma_1 V_0 \psi\|^2 + \cdots + \|\Gamma_{L-1} V_{L-2} \cdots V_1 V_0 \psi\|^2 \quad (59)$$

$$\leq (L-1) \|\psi\|^2 + \sum_{j=0}^{L-1} \|\Gamma_j\|^2 \|\psi\|^2, \quad (60)$$

thus

$$w(S) = \|\Gamma\|^2 \leq L-1 + \sum_{j=0}^{L-1} w(S_j). \quad (61)$$

To estimate $K(S) = \|(I - T)^{-1}\|$, let us solve the linear system

$$(I - T) \begin{pmatrix} h_1 \\ h_2 \\ \vdots \\ h_{L-1} \\ v_0 \\ v_1 \\ v_2 \\ \vdots \\ v_{L-1} \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_{L-1} \\ b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_{L-1} \end{pmatrix} =: g \quad (62)$$

for arbitrary vectors a_j and b_j . Then we have the recursive relation (with supplementary definition $h_0 = 0$)

$$h_j - D_{j-1} h_{j-1} - E_{j-1} v_{j-1} = a_j \quad (63)$$

$$-B_j h_j + (I - T_j) v_j = b_j. \quad (64)$$

Using Lemma 2.7, Eq. (64) becomes

$$v_j = \Gamma_j h_j + (I - T_j)^{-1} b_j. \quad (65)$$

Substituting this into Eq. (63) gives

$$h_j = D_{j-1} h_{j-1} + E_{j-1} (\Gamma_{j-1} h_{j-1} + (I - T_{j-1})^{-1} b_{j-1}) + a_j \quad (66)$$

$$= (D_{j-1} + E_{j-1} \Gamma_{j-1}) h_{j-1} + a_j + E_{j-1} (I - T_{j-1})^{-1} b_{j-1}. \quad (67)$$

Therefore, according to Lemma 2.7 again,

$$h_j = V_{j-1} h_{j-1} + a_j + V_{j-1} \Gamma_{j-1}^\dagger b_{j-1}. \quad (68)$$

Solving this recursive relation gives a block matrix form $h = \mathcal{L}f$, where $h = [h_1; h_2; \dots; h_{L-1}]$, $f = [f_1; f_2; \dots; f_{L-1}]$ with

$$f_j = a_j + V_{j-1} \Gamma_{j-1}^\dagger b_{j-1}, \quad (69)$$

and

$$\mathcal{L} = \begin{pmatrix} I & 0 & 0 & \cdots & 0 \\ V_1 & I & 0 & \cdots & 0 \\ V_2 V_1 & V_2 & I & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ V_{L-2} \cdots V_1 & V_{L-2} \cdots V_2 & \cdots & V_{L-2} & I \end{pmatrix}. \quad (70)$$

Notice that $\|\mathcal{L}\| \leq L - 1$ and $\|f_j\| \leq \|a_j\| + \|\Gamma_{j-1}\| \|b_{j-1}\|$, we can bound

$$\|h\|^2 \leq \|\mathcal{L}\|^2 \|f\|^2 = \|\mathcal{L}\|^2 \sum_{j=1}^{L-1} \|f_j\|^2 \quad (71)$$

$$\leq (L-1)^2 \sum_{j=1}^{L-1} (\|a_j\| + \|\Gamma_{j-1}\| \|b_{j-1}\|)^2 \quad (72)$$

$$\leq (L-1)^2 \sum_{j=1}^{L-1} (1 + w(S_{j-1})) (\|a_j\|^2 + \|b_{j-1}\|^2) \quad (73)$$

$$\leq (L-1)^2 (1 + \max_j w(S_j)) \|g\|^2. \quad (74)$$

Now, using Eq. (65), we decompose

$$(I - T)^{-1} g = \begin{pmatrix} h_1 \\ \vdots \\ h_{L-1} \\ v_0 \\ v_1 \\ \vdots \\ v_{L-1} \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ (I - T_0)^{-1} b_0 \\ (I - T_1)^{-1} b_1 \\ \vdots \\ (I - T_{L-1})^{-1} b_{L-1} \end{pmatrix} + \begin{pmatrix} h_1 \\ \vdots \\ h_{L-1} \\ 0 \\ \Gamma_1 h_1 \\ \vdots \\ \Gamma_{L-1} h_{L-1} \end{pmatrix}. \quad (75)$$

Then the second term on the right hand side of Eq. (75) has norm

$$\sqrt{\|h\|^2 + \sum_{j=1}^{L-1} \|\Gamma_j h_j\|^2} \leq \sqrt{\|h\|^2 + \max_j w(S_j) \sum_{j=1}^{L-1} \|h_j\|^2} \quad (76)$$

$$= \sqrt{1 + \max_j w(S_j)} \|h\| \quad (77)$$

$$\leq (L-1) (1 + \max_j w(S_j)) \|g\|. \quad (78)$$

For the first term on the right hand side of Eq. (75), its norm is

$$\sqrt{\sum_{j=0}^{L-1} \|(I - T_j)^{-1} b_j\|^2} \leq \max_j \|(I - T_j)^{-1}\| \sqrt{\sum_{j=0}^{L-1} \|b_j\|^2} \leq \max_j K(S_j) \|g\|. \quad (79)$$

Combining these two estimates together, we have

$$\|(I - T)^{-1} g\| \leq (\max_j K(S_j) + (L - 1)(1 + \max_j w(S_j))) \|g\|, \quad (80)$$

and thus

$$K(S) = \|(I - T)^{-1}\| \leq \max_j K(S_j) + (L - 1)(1 + \max_j w(S_j)). \quad (81)$$

□

Lemma 3.2 shows how to construct a larger transducer for the sequential composition of multiple transducers. Lemma 3.2 also estimates two norms. Importantly, Eq. (46) shows that the individual resolvent contributions enter through $\max_j K(S_j)$, rather than through their sum. Later we will see that $K(S)$ is directly related to the algorithm complexity, so Eq. (46) is the key reason why transducer-based LCU can outperform the standard LCU implementation.

Although Lemma 3.2 only considers the case where all the unitaries V_j are realized through the transducers, the same result can be generalized to allow inverse, controlled version, untouched qubits, and transducer-independent gates. Specifically, if the circuit includes $V_j^\dagger$, then we can simply use $S_j^\dagger$. To see this, for any vector ψ , we define $\phi = V_j^\dagger \psi$ and using the definition of the transducer for ϕ to write

$$S_j(\phi \oplus \Gamma_j \phi) = V_j \phi \oplus \Gamma_j \phi. \quad (82)$$

Apply $S_j^\dagger$ on both sides, and we have

$$S_j^\dagger(\psi \oplus \Gamma_j V_j^\dagger \psi) = V_j^\dagger \psi \oplus \Gamma_j V_j^\dagger \psi, \quad (83)$$

i.e., $S_j^\dagger$ is a transducer of $V_j^\dagger$ with the norm $w(S_j) = \|\Gamma_j V_j^\dagger\|^2 = \|\Gamma_j\|^2$ unchanged. Furthermore, $S_j^\dagger = [D_j^\dagger, B_j^\dagger; E_j^\dagger, T_j^\dagger]$, so $K(S_j^\dagger) = \|(I - T_j^\dagger)^{-1}\| = \|(I - T_j)^{-1}\| = K(S_j)$ also remains unchanged. Another scenario is that there are additional V_j -independent gates. For example, if we would like to implement $G_j V_j$, where G_j represents a collection of known gates, then we can use $[G_j, 0; 0, I] S_j$, which is the transducer of $G_j V_j$ with catalyst and the lower right block unchanged. If G_j comes before V_j , i.e., $V_j G_j$, then we use $S_j [G_j, 0; 0, I]$, which transduces $V_j G_j$ with the same lower right block. The corresponding catalyst becomes $\Gamma_j G_j$, so its norm remains unchanged. For the controlled version of V_j , we regard the control register as part of the public space. Writing

$$S_j = \begin{pmatrix} D_j & E_j \\ B_j & T_j \end{pmatrix},$$

we may use

$$S_j^{\text{ct}} = \begin{pmatrix} I & 0 & 0 \\ 0 & D_j & E_j \\ 0 & B_j & T_j \end{pmatrix},$$

which transduces the controlled- V_j operation. Its private block remains T_j , and hence

$$w(S_j^{\text{ct}}) = w(S_j), \quad K(S_j^{\text{ct}}) = K(S_j).$$

Summarizing the above discussions, we have the following result.

Corollary 3.3 (Transducer for transducer and unitary composition). *For $1 \leq j \leq L$, let V_j be unitaries (not necessarily of the same dimension), and S_j be the transducer of V_j with catalyst map Γ_j . Let $\tilde{V}_j$, unitaries of the same dimension, represent one of the following cases: V_j itself or $V_j \otimes I$, their inverses and controlled versions. Let G_j denote unitaries independent of V_j 's. Then there exists a transducer S of $G_L \tilde{V}_L G_{L-1} \tilde{V}_{L-1} \cdots G_1 \tilde{V}_1 G_0$ such that*

$$w(S) \leq L - 1 + \sum_{j=1}^L w(S_j), \quad (84)$$

$$K(S) \leq \max_j K(S_j) + (L - 1)(1 + \max_j w(S_j)), \quad (85)$$

using one query to the select oracle of S_j .

3.2 Catalyst removal via finite reuse

We now explain the finite reuse construction that will be used to turn a transducer into a unitary implementation of its action without the need of explicit catalyst. The idea is to introduce N copies of the public space and a shared private space initialized by a zero vector input rather than the exact catalyst. The finite reuse then repeats the transducer for N times with different public spaces and the shared same private space, reusing the private output of one as the private input of the next transducer operation.

We discuss the finite reuse procedure more precisely. Let $S = [D, E; B, T]$ be a unitary transducer with action V and catalyst Γ . Introduce $\mathcal{H}_{\text{pub}}^{(0)}, \dots, \mathcal{H}_{\text{pub}}^{(N-1)} \simeq \mathcal{H}_{\text{pub}}$, and the finite reuse circuit acts on $\left(\bigoplus_{j=0}^{N-1} \mathcal{H}_{\text{pub}}^{(j)}\right) \oplus \mathcal{H}_{\text{priv}}$. For an arbitrary input vector ψ , we first append ancilla qubits to get $(|0\rangle \otimes \psi) \oplus \mathbf{0}$, where the first register consists of $\log N$ ancilla qubits, and $\mathbf{0} = [0; 0; \dots; 0]$ represents the zero vector. We apply the Hadamard gates to prepare a uniform superposition over the first ancilla register, and the state becomes

$$\left(\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle \otimes \psi\right) \oplus \mathbf{0} = [\psi/\sqrt{N}; \dots; \psi/\sqrt{N}; \mathbf{0}] \in \left(\bigoplus_{j=0}^{N-1} \mathcal{H}_{\text{pub}}^{(j)}\right) \oplus \mathcal{H}_{\text{priv}}. \quad (86)$$

Then we apply S for N times, sequentially on the space $\mathcal{H}_{\text{pub}}^{(j)} \oplus \mathcal{H}_{\text{priv}}$ for $0 \leq j \leq N - 1$. To describe this step explicitly, let q_j denote the private vector immediately before the j -th application of S (and thus $q_0 = \mathbf{0}$), and let o_j denote the public output produced after that application. At each reuse, the public input is always $\psi/\sqrt{N}$, so we have

$$S \begin{pmatrix} \psi/\sqrt{N} \\ q_j \end{pmatrix} = \begin{pmatrix} o_j \\ q_{j+1} \end{pmatrix}, \quad (87)$$

i.e.,

$$o_j = \frac{1}{\sqrt{N}} D\psi + E q_j, \quad q_{j+1} = \frac{1}{\sqrt{N}} B\psi + T q_j, \quad 0 \leq j \leq N - 1. \quad (88)$$

Notice that both o_j and q_j can be written as a linear transformation of ψ . After all the N applications of S , the entire output vector becomes

$$\left(\sum_{j=0}^{N-1} |j\rangle \otimes o_j\right) \oplus q_N. \quad (89)$$

Applying Hadamard gates on the first register again yields the output like

$$\left(|0\rangle \otimes P_N \psi + \sum_{j=1}^{N-1} |j\rangle \otimes *\right) \oplus *, \quad (90)$$

where

$$P_N\psi := \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} o_j. \quad (91)$$

Post-selecting all the ancilla qubits (including those in the first register and for private space) onto 0 gives $P_N\psi$. Therefore, the pre-described circuit constructs a block-encoding of P_N .

Since the private space is not the ideal catalyst, P_N is thereby not the ideal action V . The following lemma identifies the error incurred by starting with zero private input.

Lemma 3.4 (Reuse error identity [23]). *Let S be a transducer with action V and catalyst Γ . Let P_N be the upper left block of the length- N finite reuse circuit, defined by Eq. (91). Then*

$$V - P_N = EG_N(T)\Gamma, \quad G_N(z) = \frac{1}{N} \sum_{k=0}^{N-1} z^k. \quad (92)$$

Proof. Recall that for any input vector ψ , $P_N\psi = (\sum_{j=0}^{N-1} o_j)/\sqrt{N}$ where o_j 's are given by the recursion Eq. (88). Solving the linear recursion for q_j with $q_0 = \mathbf{0}$ yields

$$q_j = \frac{1}{\sqrt{N}}(I + T + T^2 + \dots + T^{j-1})B\psi. \quad (93)$$

According to Lemma 2.7, we have $B = (I - T)\Gamma$, and thus

$$q_j = \frac{1}{\sqrt{N}}(I + T + T^2 + \dots + T^{j-1})(I - T)\Gamma\psi = \frac{1}{\sqrt{N}}(I - T^j)\Gamma\psi. \quad (94)$$

Substituting it into the first equation of Eq. (88) gives

$$o_j = \frac{1}{\sqrt{N}}D\psi + E\frac{1}{\sqrt{N}}(I - T^j)\Gamma\psi = \frac{1}{\sqrt{N}}(D + E\Gamma - ET^j\Gamma)\psi = \frac{1}{\sqrt{N}}(V - ET^j\Gamma)\psi, \quad (95)$$

where the last equation follows from Lemma 2.7 again. Therefore

$$P_N\psi = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} o_j = \frac{1}{N} \sum_{j=0}^{N-1} (V - ET^j\Gamma)\psi = V\psi - EG_N(T)\Gamma\psi, \quad (96)$$

and thus

$$V - P_N = EG_N(T)\Gamma. \quad (97)$$

□

Lemma 3.4 already suggests that the finite reuse can turn a transducer to a block-encoding of ϵ -approximation of V for arbitrarily small ϵ , but the convergence is only first-order. Specifically, suppose that $\|T\| < 1$, then $\|G_N(T)\| = \mathcal{O}(1/N)$, and Lemma 3.4 implies $\|V - P_N\| = \mathcal{O}(1/N)$. So it suffices to choose $N = \mathcal{O}(1/\epsilon)$ for an ϵ -approximation.

To achieve high-order convergence, we can choose different values of N and linearly combine the corresponding block-encodings to cancel the low-order error terms. This is in principle the extrapolation method, which has been widely used for improved accuracy in both classical and quantum numerical algorithms. Classical examples include Richardson extrapolation for finite-difference discretisation [43] and extrapolation methods for ordinary differential equations such as the Bulirsch–Stoer method [44]. A closely related idea also appears in quantum Hamiltonian simulation through multi-product formulas, where linear combinations of product formula approximations with different step sizes are chosen to cancel low-order Trotter errors and thereby achieve higher order accuracy [45, 46], and the reduced-depth quantum algorithms for Hamiltonian simulation [47] and linear differential equations [48]. Similar idea further serves as the guiding principle for the famous zero-noise extrapolation [49, 50] in quantum error mitigation.

The idea of combining reuse circuits of different lengths was recently introduced in the context of optimal time-dependent Hamiltonian simulation [23, 24], where a carefully chosen weighted combination exploits the time-ordered structure of the private block to achieve factorial error decay. The same principle has subsequently been applied to Lindblad simulation [25]. Here we establish a structure-independent version of this idea. Rather than relying on a special form of the private block, we show that the sole assumption that $I - T$ is invertible is sufficient. We formally state the result as follows.

Theorem 3.5 (High-order finite reuse). *Let $S = [D, E; B, T]$ be a transducer of V with invertible $(I - T)$. Then, we can implement a block-encoding of $\tilde{V}$ such that $\|\tilde{V} - V\| \leq \mathcal{O}(\epsilon)$, using*

$$\mathcal{O}\left(K(S) \log\left(\frac{1}{\epsilon}\right)\right) \quad (98)$$

queries to S , its inverse, and controlled versions.

Proof. Following the notation in Lemma 3.4, let P_N denote the matrix constructed through the length- N finite reuse circuit. We consider the linear combination of finite reuse circuits with different values of N as

$$\tilde{V} = \sum_{k=0}^L b_k \left(\frac{R_0 + k}{R_0} P_{R_0+k} - \frac{k}{R_0} P_k \right). \quad (99)$$

Here b_k , L and R_0 are some parameters to be determined later, but we first suppose that $b_k \geq 0$ and $\sum b_k = 1$. For any fixed k , according to Lemma 3.4,

$$\frac{R_0 + k}{R_0} P_{R_0+k} - \frac{k}{R_0} P_k = \frac{R_0 + k}{R_0} (V - EG_{R_0+k}(T)\Gamma) - \frac{k}{R_0} (V - EG_k(T)\Gamma) \quad (100)$$

$$= V - E \left(\frac{R_0 + k}{R_0} G_{R_0+k}(T) - \frac{k}{R_0} G_k(T) \right) \Gamma \quad (101)$$

$$= V - E \left(\frac{1}{R_0} \sum_{j=k}^{R_0+k-1} T^j \right) \Gamma \quad (102)$$

$$= V - ET^k G_{R_0}(T)\Gamma. \quad (103)$$

Then

$$\tilde{V} = \sum_{k=0}^L b_k V - \sum_{k=0}^L b_k ET^k G_{R_0}(T)\Gamma = V - E\Phi(T)G_{R_0}(T)\Gamma, \quad (104)$$

where we have used $\sum b_k = 1$ and denote

$$\Phi(z) = \sum_{k=0}^L b_k z^k. \quad (105)$$

To bound this error, we first rewrite

$$G_{R_0}(T) = \frac{1}{R_0} \sum_{k=0}^{R_0-1} T^k = \frac{1}{R_0} (I - T)^{-1} (I - T^{R_0}). \quad (106)$$

Notice that the matrices $(I - T)^{-1}$, $\Phi(T)$, $I - T^{R_0}$ commute with one another. Hence

$$V - \tilde{V} = \frac{1}{R_0} E\Phi(T)(I - T)^{-1}(I - T^{R_0})\Gamma \quad (107)$$

$$= \frac{1}{R_0} E(I - T)^{-1}(I - T^{R_0})\Phi(T)\Gamma \quad (108)$$

$$= \frac{1}{R_0} V\Gamma^\dagger(I - T^{R_0})\Phi(T)\Gamma \quad (109)$$

where the last line follows from Lemma 2.7. Taking the norm, we have

$$\|V - \tilde{V}\| \leq \frac{w(S)}{R_0} \|I - T^{R_0}\| \|\Phi(T)\|. \quad (110)$$

In order to control the error, we need to construct proper $\Phi(T)$ with sufficiently small norm. Notice that $\Phi(1) = 1$ due to $\sum b_k = 1$, so we have to bound T away from 1, which, to some extent, can be achieved by leveraging the invertible assumption of $(I - T)$. Motivated by this, the idea of the construction is to choose⁵

$$\Phi(z) = (G_{N_0}(z))^q \quad (111)$$

for some integers N_0 and q (so the degree $L = q(N_0 - 1)$), select N_0 such that $\|G_{N_0}(T)\|$ (which depends on $\|(I - T)^{-1}\|$) is strictly smaller than 1, and choose q to be logarithmic in $1/\epsilon$ to achieve the desired accuracy. Specifically, using $G_{N_0}(T) = \frac{1}{N_0}(I - T)^{-1}(I - T^{N_0})$ again, and noticing that $\|T\| \leq 1$ since it is a sub-block of the unitary S , we have

$$\|G_{N_0}(T)\| \leq \frac{2}{N_0} K(S) \leq \frac{1}{2}, \quad (112)$$

by choose

$$N_0 = \lceil 4K(S) \rceil. \quad (113)$$

Then $\|\Phi(T)\| = \|(G_{N_0}(T))^q\| \leq 2^{-q}$, and thus Eq. (110) becomes

$$\|V - \tilde{V}\| \leq \frac{2w(S)}{R_0} \frac{1}{2^q}. \quad (114)$$

To force the right hand side to be smaller than ϵ , it suffices to choose

$$q = \left\lceil \log_2 \left(\frac{1}{\epsilon} \right) \right\rceil, \quad L = q(N_0 - 1) = \left\lceil \log_2 \left(\frac{1}{\epsilon} \right) \right\rceil (\lceil 4K(S) \rceil - 1), \quad R_0 = 2 \left\lceil \log_2 \left(\frac{1}{\epsilon} \right) \right\rceil \lceil 4K(S) \rceil. \quad (115)$$

Then $2^{-q} \leq \epsilon$, and $R_0 \geq 2(2K(S) - 1) \geq 2w(S)$ according to Lemma 2.7, which together imply $\|V - \tilde{V}\| \leq \epsilon$. Notice that R_0 does not need to be as large as $\mathcal{O}(L)$ if we only wish to bound the error by ϵ , but such a larger choice is necessary to ensure the normalization factor of the LCU implementation bounded, as we will discussed next.

To implement Eq. (99), we can directly apply standard LCU in Lemma 2.3, which gives a block-encoding of $\tilde{V}/\alpha$ with normalization factor

$$\alpha = \sum_{k=0}^L b_k \left(\frac{R_0 + k}{R_0} + \frac{k}{R_0} \right) = \sum_{k=0}^L b_k \frac{R_0 + 2k}{R_0} \leq 1 + \frac{2L}{R_0} < 2, \quad (116)$$

where the last inequality is $R_0 \geq 2L$, guaranteed by the choice of R_0 and L in Eq. (115). We relax this normalization factor to exactly 2 by append an additional ancilla qubit and perform a rotation gate. Then, we apply the robust oblivious amplitude amplification technique [51] using 3 calls to the LCU circuit to boost the normalization factor to 1 with block-encoding error being $\mathcal{O}(\epsilon)$.

This procedure takes $\mathcal{O}(1)$ queries to the select oracle of P_j 's block-encodings. As, according to Lemma 3.4, the block-encoding of P_j is constructed mainly by the sequentially application of a single transducer S for j times, the select oracle of P_j 's block-encodings can be constructed by imposing proper control logic on the most expensive P_j , which is P_{R_0+L} . This requires $\mathcal{O}(R_0 + L)$ calls to S , which yields the desired complexity according to the choice of R_0 and L in Eq. (115). $\square$

⁵Notice that $G_{N_0}(1) = 1$ and thus $\Phi(1) = 1$ for any N_0 and q , and all the coefficients of $G_{N_0}(z)$ are positive. Hence the conditions $b_k \geq 0$ and $\sum b_k = 1$ are automatically satisfied.

Remark 3.6 (Comparison with Lemma 2.6). *For a fixed input ψ , the standard transducer implementation achieves state error ϵ using*

$$\mathcal{O}\left(1 + \frac{W(S, \psi)}{\epsilon^2}\right) \quad (117)$$

controlled applications of S . In particular, using $W(S, \psi) \leq w(S)$ gives a worst-case bound $\mathcal{O}(1 + w(S)/\epsilon^2)$. By contrast, Theorem 3.5 gives an operator-norm approximation to the entire public action using

$$\mathcal{O}\left(K(S) \log \frac{1}{\epsilon}\right) \quad (118)$$

queries, provided a bound on $K(S) = \|(I - T)^{-1}\|$ is available. Thus, for a fixed transducer with controlled resolvent norm, the dependence on the target precision improves from inverse polynomial to logarithmic. This is not a uniform improvement in the transducer parameter: by Lemma 2.7, we have $w(S) \leq 2K(S) - 1$, and $K(S)$ can in general be substantially larger than $w(S)$. The advantage of the high-order construction is therefore most useful when the resolvent norm can be bounded comparably to, or only mildly worse than, the catalyst complexity, as occurs in the constructions used in this work.

4 Transducer-based linear combination of unitaries algorithm

Now we are ready to establish our transducer-based LCU algorithm. We first discuss an important subroutine that constructs a transducer of the Cayley transformation of LCU with normalized coefficients. Then we present the entire transducer-based LCU algorithm and analyze its complexity.

4.1 Cayley transformation of LCU with normalized coefficients

In this subsection, let V_j be a Hermitian block-encoding of H_j , and suppose that V_j satisfies Assumption 2.1. We consider the linear combination

$$Y = \sum_{j=0}^{J-1} q_j H_j, \quad (119)$$

where we assume $q_j \geq 0$ and $\sum q_j = 1$. Let V be the select oracle of V_j , i.e.,

$$V = \sum_{j=0}^{J-1} |j\rangle \langle j| \otimes V_j, \quad (120)$$

and U_p prepares the coefficient state in the sense that

$$U_p |0\rangle = \sum_{j=0}^{J-1} \sqrt{q_j} |j\rangle. \quad (121)$$

The subroutine is as follows. The full space consists of four registers: the system register which is used for the data, the signal register which serves as the ancilla register of the block-encoding, the index register for j in the linear combination, and the clock register introduced by the transducer. We first apply Lemma 3.1 to construct a transducers $S_{V_j} = [0, E_{V_j}; B_{V_j}, T_{V_j}]$ of V_j , then consider $\oplus S_{V_j}$ with reordering all the public sectors first then followed by private sections to get S_V . Specifically, $S_V = [0, E_V; B_V, T_V]$ where $E_V = \text{diag}(E_{V_0}, \dots, E_{V_{J-1}})$, $B_V = \text{diag}(B_{V_0}, \dots, B_{V_{J-1}})$, and $T_V = \text{diag}(T_{V_0}, \dots, T_{V_{J-1}})$. Then we construct

$$S = \Phi U_p^\dagger S_V U_p. \quad (122)$$

Here

$$\mathcal{U}_p = |0\rangle\langle 0| \otimes U_p \otimes I_{\text{sig,sys}} + \sum_{k \neq 0} |k\rangle\langle k| \otimes I_{\text{ind,sig,sys}}, \quad (123)$$

and, using the same block division as in S_V ,

$$\Phi = \begin{pmatrix} \Phi_0 & 0 \\ 0 & I \end{pmatrix}, \quad \Phi_0 = i(I - 2|0\rangle\langle 0| \otimes |0\rangle\langle 0| \otimes I). \quad (124)$$

Roughly speaking, Φ appends $-i$ when all the ancilla registers are 0, i when the clock register is zero but the index and signal are not all zero, and does not append any factor otherwise. If we declare the space with all zero ancilla registers (but excluding the ancilla) to be the public space, then Eq. (122) is a transducer of $\text{Cay}(Y)$, as shown in the next result. Here the Cayley transformation is defined as

$$\text{Cay}(Y) = (I - iY)(I + iY)^{-1}. \quad (125)$$

Theorem 4.1 (Cayley transformation of LCU). *Let V_j be a Hermitian block-encoding of H_j , and suppose that V_j satisfies Assumption 2.1 with C_j oracle queries. Let $Y = \sum_{j=0}^{J-1} q_j H_j$ with $q_j \geq 0$ and $\sum q_j = 1$. Then, the circuit S defined in Eq. (122) gives a transducer of $\text{Cay}(Y)$ with catalyst Γ , such that*

$$\Gamma^\dagger \Gamma = 2(1 + \sum q_j C_j)(I + Y^2)^{-1} - I, \quad (126)$$

$$w(S) \leq 1 + 2 \sum q_j C_j, \quad K(S) \leq 6 \max C_j, \quad (127)$$

using $\mathcal{O}(1)$ queries to the primitive oracles.

Proof. We first define some notations which will be used through this proof. Suppose the dimension of H_j is d . Let

$$V = \sum_{j=0}^{J-1} |j\rangle\langle j| \otimes V_j, \quad (128)$$

and

$$\mathcal{J}_0 = |0\rangle_{\text{sig}} \otimes I_{\text{sys}}, \quad \mathcal{J} = \sum_{j=0}^{J-1} \sqrt{q_j} |j\rangle \otimes \mathcal{J}_0 = (U_p \otimes I)(|0\rangle \otimes \mathcal{J}_0). \quad (129)$$

Then

$$(U_p \otimes I_{\text{sig,sys}})(|0\rangle|0\rangle \otimes I) = (U_p|0\rangle)|0\rangle \otimes I = \mathcal{J}, \quad (130)$$

i.e., the first d column of $U_p \otimes I_{\text{sig,sys}}$ is $\mathcal{J}$. Then we can write the full representation of $U_p \otimes I_{\text{sig,sys}}$ as

$$U_p \otimes I_{\text{sig,sys}} = \begin{pmatrix} \mathcal{J} & \mathcal{J}_\perp \end{pmatrix}, \quad (131)$$

where $\mathcal{J}^\dagger \mathcal{J}_\perp = 0$, $\mathcal{J}_\perp^\dagger \mathcal{J}_\perp = I$. Additionally, by the standard LCU calculation, we have

$$\mathcal{J}^\dagger V \mathcal{J} = Y. \quad (132)$$

The algorithm first applies Lemma 3.1 to construct transducers $S_{V_j} = [0, E_{V_j}; B_{V_j}, T_{V_j}]$ of V_j using $\mathcal{O}(1)$ queries, then construct $S_V = [0, E_V; B_V, T_V]$ where $E_V = \text{diag}(E_{V_0}, \dots, E_{V_{J-1}})$, $B_V = \text{diag}(B_{V_0}, \dots, B_{V_{J-1}})$, and $T_V = \text{diag}(T_{V_0}, \dots, T_{V_{J-1}})$. Since S_V is basically a block reordering of the parallel controlled applications of S_{V_j} , constructing of S_V can be also achieved using $\mathcal{O}(1)$ queries. Therefore, the query complexity of S defined in Eq. (122) is $\mathcal{O}(1)$.

Using the block representations

$$S_V = \begin{pmatrix} 0 & E_V \\ B_V & T_V \end{pmatrix}, \quad \mathcal{U}_p = \begin{pmatrix} \mathcal{J} & \mathcal{J}_\perp & 0 \\ 0 & 0 & I \end{pmatrix}, \quad \Phi = \begin{pmatrix} -iI & & \\ & iI & \\ & & I \end{pmatrix}, \quad (133)$$

then we have

$$S = \begin{pmatrix} 0 & 0 & -i\mathcal{J}^\dagger E_V \\ 0 & 0 & i\mathcal{J}_\perp^\dagger E_V \\ B_V \mathcal{J} & B_V \mathcal{J}_\perp & T_V \end{pmatrix} =: \begin{pmatrix} 0 & E \\ B & T \end{pmatrix}, \quad (134)$$

where

$$E = \begin{pmatrix} 0 & -i\mathcal{J}^\dagger E_V \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ B_V \mathcal{J} \end{pmatrix}, \quad T = \begin{pmatrix} 0 & i\mathcal{J}_\perp^\dagger E_V \\ B_V \mathcal{J}_\perp & T_V \end{pmatrix}. \quad (135)$$

Now let us verify that S is the transducer of $\text{Cay}(Y)$. To this end, for any vector ψ , let us solve the equation

$$S \begin{pmatrix} \psi \\ c \\ h \end{pmatrix} = \begin{pmatrix} \tilde{V}\psi \\ c \\ h \end{pmatrix} \quad (136)$$

where $\tilde{V}$ is the unknown action matrix, and c and h are supposed to be the linear transformation of ψ . Then we have

$$\tilde{V}\psi = -i\mathcal{J}^\dagger E_V h \quad (137)$$

$$c = i\mathcal{J}_\perp^\dagger E_V h \quad (138)$$

$$h = B_V(\mathcal{J}\psi + \mathcal{J}_\perp c) + T_V h. \quad (139)$$

Let

$$y = \mathcal{J}\psi + \mathcal{J}_\perp c, \quad (140)$$

then we have

$$h = (I - T_V)^{-1} B_V y, \quad (141)$$

and thus

$$c = i\mathcal{J}_\perp^\dagger V y = iX^\dagger \psi + iZc, \quad \tilde{V}\psi = -i\mathcal{J}^\dagger V y = -iY\psi - iXc, \quad (142)$$

where we use $E_V(I - T_V)^{-1} B_V = V$, which is a direct consequence of $E_{V_j}(I - T_{V_j})^{-1} B_{V_j} = V_j$ from Lemma 3.1, and further denote

$$X = \mathcal{J}^\dagger V \mathcal{J}_\perp, \quad Z = \mathcal{J}_\perp^\dagger V \mathcal{J}_\perp. \quad (143)$$

To further solve Eq. (141) and Eq. (142), we need to note that $(U_p \otimes I)^\dagger V (U_p \otimes I) = [Y, X; X^\dagger, Z]$ is a Hermitian unitary and thus

$$XX^\dagger = I - Y^2, \quad X^\dagger Y = -ZX^\dagger. \quad (144)$$

Then the first equation of Eq. (142) becomes

$$c = i(I - iZ)^{-1} X^\dagger \psi = iX^\dagger (I + iY)^{-1} \psi, \quad (145)$$

where the last equation can be verified using $X^\dagger Y = -ZX^\dagger$. Plugging Eq. (145) back to the second equation in Eq. (142) gives

$$\tilde{V}\psi = -iY\psi + XX^\dagger (I + iY)^{-1} \psi \quad (146)$$

$$= -iY\psi + (I - Y^2)(I + iY)^{-1} \psi \quad (147)$$

$$= (-iY(I + iY) + I - Y^2)(I + iY)^{-1} \psi \quad (148)$$

$$= (I - iY)(I + iY)^{-1} \psi, \quad (149)$$

so the action $\tilde{V} = \text{Cay}(Y)$. Using Eq. (140), Eq. (141) and Eq. (145), we can also write

$$y = \mathcal{J}\psi + i\mathcal{J}_\perp X^\dagger(I + iY)^{-1}\psi \quad (150)$$

$$= (\mathcal{J} + i(\mathcal{J}Y + \mathcal{J}_\perp X^\dagger))(I + iY)^{-1}\psi \quad (151)$$

$$= (\mathcal{J} + i(\mathcal{J}\mathcal{J}^\dagger V\mathcal{J} + \mathcal{J}_\perp \mathcal{J}_\perp^\dagger V\mathcal{J}))(I + iY)^{-1}\psi \quad (152)$$

$$= (I + iV)\mathcal{J}(I + iY)^{-1}\psi, \quad (153)$$

and thus

$$h = (I - T_V)^{-1}B_V(I + iV)\mathcal{J}(I + iY)^{-1}\psi. \quad (154)$$

Then the catalyst map is given as

$$\Gamma\psi = \begin{pmatrix} c \\ h \end{pmatrix} = \begin{pmatrix} iX^\dagger(I + iY)^{-1} \\ (I - T_V)^{-1}B_V(I + iV)\mathcal{J}(I + iY)^{-1} \end{pmatrix} \psi. \quad (155)$$

Finally, let us derive the expression of $\Gamma^\dagger\Gamma$ and bound the related norms. We start with y defined in Eq. (140) and write

$$y = \sum_{j=0}^{J-1} |j\rangle \otimes y_j. \quad (156)$$

Then each component y_j is

$$y_j = (\langle j| \otimes I)y = (\langle j| \otimes I)(I + iV)\mathcal{J}(I + iY)^{-1}\psi \quad (157)$$

$$= (\langle j| \otimes I)(I + iV) \left(\sum_{j'=0}^{J-1} \sqrt{q_{j'}} |j'\rangle \mathcal{J}_0 \right) (I + iY)^{-1}\psi \quad (158)$$

$$= \sqrt{q_j}(I + iV_j)\mathcal{J}_0(I + iY)^{-1}\psi. \quad (159)$$

Since V_j is Hermitian unitary, we have $(I + iV_j)^\dagger(I + iV_j) = 2I$, and thus

$$\|y_j\|^2 = 2q_j\|(I + iY)^{-1}\psi\|^2, \quad \|y\|^2 = \sum \|y_j\|^2 = 2\|(I + iY)^{-1}\psi\|^2. \quad (160)$$

Notice that we have $h = (I - T_V)^{-1}B_V y = \sum_{j=0}^{J-1} |j\rangle \otimes (I - T_{V_j})^{-1}B_{V_j}y_j$, and, according to Lemma 3.1 that $\|(I - T_{V_j})^{-1}B_{V_j}y_j\|^2 = C_j\|y_j\|^2$, we have

$$\|h\|^2 = \|(I - T_V)^{-1}B_V y\|^2 = \sum_{j=0}^{J-1} C_j\|y_j\|^2 = 2 \left(\sum_{j=0}^{J-1} q_j C_j \right) \|(I + iY)^{-1}\psi\|^2. \quad (161)$$

Also

$$\|(I + iY)^{-1}\psi\|^2 = \psi^\dagger(I - iY^\dagger)^{-1}(I + iY)^{-1}\psi = \psi^\dagger(I + Y^2)^{-1}\psi, \quad (162)$$

and then (using $y = \mathcal{J}\psi + \mathcal{J}_\perp c$ to be an orthogonal decomposition)

$$\|c\|^2 = \|y\|^2 - \|\psi\|^2 = 2\|(I + iY)^{-1}\psi\|^2 - \|\psi\|^2. \quad (163)$$

Therefore

$$\psi^\dagger\Gamma^\dagger\Gamma\psi = \|\Gamma\psi\|^2 = \|c\|^2 + \|h\|^2 = 2(1 + \sum q_j C_j)\|(I + iY)^{-1}\psi\|^2 - \|\psi\|^2 \quad (164)$$

$$= \psi^\dagger \left(2(1 + \sum q_j C_j)(I + Y^2)^{-1} - I \right) \psi, \quad (165)$$

which implies that

$$\Gamma^\dagger\Gamma = 2(1 + \sum q_j C_j)(I + Y^2)^{-1} - I, \quad (166)$$

and (using $(I + Y^2)^{-1} \preceq I$)

$$w(S) = \|\Gamma\|^2 = \sup_{\|\psi\|=1} \psi^\dagger \left(2(1 + \sum q_j C_j)(I + Y^2)^{-1} - I \right) \psi \leq 1 + 2 \sum q_j C_j. \quad (167)$$

To bound $K(S)$, from Eq. (135) we write

$$I - T = \begin{pmatrix} I & -i\mathcal{J}_\perp^\dagger E_V \\ -B_V \mathcal{J}_\perp & I - T_V \end{pmatrix}. \quad (168)$$

To invert it, let us solve $(I - T)[x_0; x_1] = [z_0; z_1]$, i.e.,

$$x_0 - i\mathcal{J}_\perp^\dagger E_V x_1 = z_0 \quad (169)$$

$$-B_V \mathcal{J}_\perp x_0 + (I - T_V)x_1 = z_1. \quad (170)$$

Plugging the second equation back to the first equation gives

$$x_0 - i\mathcal{J}_\perp^\dagger E_V (I - T_V)^{-1} (B_V \mathcal{J}_\perp x_0 + z_1) = z_0, \quad (171)$$

which, using $E_V(I - T_V)^{-1} B_V = V$, becomes

$$x_0 = (I - iZ)^{-1} z_0 + i(I - iZ)^{-1} \mathcal{J}_\perp^\dagger E_V (I - T_V)^{-1} z_1. \quad (172)$$

The x_1 component becomes

$$x_1 = (I - T_V)^{-1} B_V \mathcal{J}_\perp x_0 + (I - T_V)^{-1} z_1 \quad (173)$$

$$= (I - T_V)^{-1} B_V \mathcal{J}_\perp (I - iZ)^{-1} z_0 \quad (174)$$

$$+ i(I - T_V)^{-1} B_V \mathcal{J}_\perp (I - iZ)^{-1} \mathcal{J}_\perp^\dagger E_V (I - T_V)^{-1} z_1 + (I - T_V)^{-1} z_1. \quad (175)$$

The expressions of x_0 and x_1 means that $(I - T)^{-1}$ equals

$$\begin{pmatrix} (I - iZ)^{-1} & i(I - iZ)^{-1} \mathcal{J}_\perp^\dagger E_V (I - T_V)^{-1} \\ (I - T_V)^{-1} B_V \mathcal{J}_\perp (I - iZ)^{-1} & (I - T_V)^{-1} + i(I - T_V)^{-1} B_V \mathcal{J}_\perp (I - iZ)^{-1} \mathcal{J}_\perp^\dagger E_V (I - T_V)^{-1} \end{pmatrix}. \quad (176)$$

Since $((I - iZ)^{-1})^\dagger (I - iZ)^{-1} = (I + Z^2)^{-1} \preceq I$ and $\|(I - iZ)^{-1}\| \leq 1$, and according to Lemma 3.1,

$$\|(I - T_V)^{-1} B_V\| \leq \max \|(I - T_{V_j})^{-1} B_{V_j}\| = \max \sqrt{C_j}, \quad (177)$$

$$\|E_V (I - T_V)^{-1}\| \leq \max \|E_{V_j} (I - T_{V_j})^{-1}\| = \max \sqrt{C_j}, \quad (178)$$

$$\|(I - T_V)^{-1}\| \leq \max \|(I - T_{V_j})^{-1}\| \leq \max C_j, \quad (179)$$

the spectral norms of the four blocks of $(I - T)^{-1}$ are bounded by 1, $\max \sqrt{C_j}$, $\max \sqrt{C_j}$, and $2 \max C_j$, respectively. Then

$$K(S) = \|(I - T)^{-1}\| \leq 1 + 2 \max \sqrt{C_j} + 2 \max C_j \leq 6 \max C_j. \quad (180)$$

□

4.2 Algorithm description

We first introduce or recall a few notations:

$$\hat{A}_j = \frac{A_j}{\alpha_j}, \quad \lambda = \sum_{j=0}^{J-1} c_j \alpha_j, \quad p_j = \frac{c_j \alpha_j}{\lambda}, \quad \bar{C} = \sum_{j=0}^{J-1} p_j C_j, \quad C_{\max} = \max_j C_j. \quad (181)$$

Consider the linear combination

$$A = \sum_{j=0}^{J-1} c_j A_j, \quad \|A\| \leq a \quad (182)$$

where a is a known upper bound of $\|A\|$, and we assume $a \leq \lambda$ as λ is the naive upper bound. The full transducer-based LCU algorithm is described as follows.

Step 1: Construct the Cayley LCU transducer. In this step we will construct a transducer of

$$\text{Cay}\left(\frac{A}{4\lambda}\right), \quad \text{where} \quad \frac{A}{4\lambda} = \sum_{j=0}^{J-1} \frac{p_j}{4} \hat{A}_j. \quad (183)$$

Notice that the coefficients $q_j = p_j/4$ only sum to $1/4$ rather than 1 , so we append $q_J = 3/4$ and V_J be any Hermitian block-encoding of a zero matrix. Then we apply the algorithm described in Section 4.1, which gives a transducer of $\text{Cay}(\sum_{j=0}^{J-1} q_j \hat{A}_j) = \text{Cay}(A/(4\lambda))$ according to Theorem 4.1.

Step 2: Perform inverse Cayley transformation and reduce the normalization factor.

Let $Y = A/(4\lambda)$ and $V = \text{Cay}(Y)$. The main idea of this step is to write down a circuit involving V to construct a block-encoding of $A/(16a)$, and use Corollary 3.3 to construct a transducer of this circuit rather than directly implementing it.

We first write down the circuit expression, which is a QSVT circuit on a small ordinary LCU. Specifically, the standard LCU can block encode

$$Z = \frac{V^\dagger - V}{2i} = 2Y(I + Y^2)^{-1}. \quad (184)$$

Notice that $\|Y\| \leq a/(4\lambda)$, we choose a polynomial $P(x)$ such that

$$P\left(\frac{2b}{1+b^2}\right) \approx \frac{b}{16(a/(4\lambda))}, \quad |b| \leq \frac{a}{4\lambda}. \quad (185)$$

Such a polynomial simultaneously inverts the Cayley transformation and reduces the normalization factor of A from $\mathcal{O}(\lambda)$ to $\mathcal{O}(a)$. A specific construction of this polynomial is given in Section A. Then the QSVT circuit with block-encoding of Z yields a block-encoding of $\frac{Y}{16(a/(4\lambda))} = A/(16a)$.

For the actual implementation, since we have constructed a transducer of V in step 1, we can apply Corollary 3.3 to turn the pre-described circuit into a transducer of the full QSVT unitary U_P , where $(\langle 0^r | \otimes I) U_P (|0^r\rangle \otimes I) = \tilde{A} \approx \frac{A}{16a}$.

Step 3: Convert the transducer to a block-encoding via finite reuse. In step 2, we have constructed a transducer of U_P . Now we directly apply Theorem 3.5 to construct a block-encoding of approximately $A/(16a)$.

4.3 Complexity analysis

Theorem 4.2 (Complexity estimate of the transducer-based LCU). *Let U_j be a Hermitian block-encoding of A_j/α_j with normalization factor α_j , and suppose that U_j satisfies Assumption 2.1 with query complexity C_j . Let $A = \sum_{j=0}^{J-1} c_j A_j$ with coefficients $c_j > 0$, and suppose that an upper bound $a \geq \|A\|$ is known. Then, for any $\epsilon > 0$, the algorithm described in Section 4.2 gives a block-encoding of A' such that $\|A' - A/(16a)\| \leq \mathcal{O}(\epsilon)$, using*

$$\mathcal{O}\left(C_{\max} \log\left(\frac{1}{\epsilon}\right) + \frac{\lambda}{a} \overline{C} \left(\log\left(\frac{1}{\epsilon}\right)\right)^2\right) \quad (186)$$

queries to the primitive oracles, where

$$\lambda = \sum_{j=0}^{J-1} c_j \alpha_j, \quad C_{\max} = \max_j C_j, \quad \overline{C} = \frac{\sum_{j=0}^{J-1} c_j \alpha_j C_j}{\lambda}. \quad (187)$$

Proof. The proof can be completed by walking through the algorithm and combining all the previously established complexity estimates for intermediate subroutine.

In Step 1, we construct a transducer S_{Cay} of $\text{Cay}(A/(4\lambda))$. Theorem 4.1 tells that

$$w(S_{\text{Cay}}) \leq 1 + 2 \sum_{j=0}^{J-1} \frac{c_j \alpha_j}{4\lambda} C_j = \mathcal{O}(\overline{C}), \quad K(S_{\text{Cay}}) \leq 6 \max_j C_j = \mathcal{O}(C_{\max}), \quad (188)$$

and the construction of S_{Cay} needs $\mathcal{O}(1)$ query complexity.

The QSVT circuit in Step 2 has query length to be $\mathcal{O}(\deg(P(x)))$, according to Lemma 2.4, where $P(x)$ is the polynomial for inverse Cayley transformation with amplification. By Lemma A.2, for any $0 < \delta_1 < 1/2$ to be determined later (and choosing $\rho = a/(4\lambda)$ in Lemma A.2), such a QSVT circuit gives a block-encoding of $\tilde{A}$ such that

$$\left\| \tilde{A} - \frac{A}{16a} \right\| \leq \delta_1, \quad (189)$$

with

$$\deg(P(x)) = \mathcal{O}\left(\frac{\lambda}{a} \log\left(\frac{1}{\delta_1}\right)\right). \quad (190)$$

Using Corollary 3.3, we can construct a transducer S of a block-encoding of $\tilde{A}$ with

$$K(S) \leq \mathcal{O}(K(S_{\text{Cay}}) + \deg(P(x))w(S_{\text{Cay}})) = \mathcal{O}\left(C_{\max} + \frac{\lambda}{a} \overline{C} \log\left(\frac{1}{\delta_1}\right)\right), \quad (191)$$

and the query complexity of S is still $\mathcal{O}(1)$.

The last step implements a finite reuse of S . For any $\delta_2 > 0$, according to Theorem 3.5, we can implement a unitary U , which is a block-encoding of $\tilde{A}$ such that

$$\|\tilde{\tilde{A}} - \tilde{A}\| \leq \mathcal{O}(\delta_2), \quad (192)$$

using

$$\mathcal{O}\left(K(S) \log\left(\frac{1}{\delta_2}\right)\right) = \mathcal{O}\left(C_{\max} \log\left(\frac{1}{\delta_2}\right) + \frac{\lambda}{a} \overline{C} \log\left(\frac{1}{\delta_1}\right) \log\left(\frac{1}{\delta_2}\right)\right) \quad (193)$$

queries to S , which turns to asymptotically the same number of queries to the primitive oracles as each S requires $\mathcal{O}(1)$ queries. As $\|\tilde{\tilde{A}} - \tilde{A}\| \leq \mathcal{O}(\delta_2)$ and $\left\| \tilde{A} - \frac{A}{16a} \right\| \leq \delta_1$, we can choose

$$\delta_1 = \Theta(\epsilon), \quad \delta_2 = \Theta(\epsilon) \quad (194)$$

to make sure that this unitary U is a block-encoding of $A/(16a)$ to error at most $\mathcal{O}(\epsilon)$. Plugging the choice of δ_1 and δ_2 back to Eq. (193) completes the proof. $\square$

5 Applications to sparse matrix functions

In this section, we discuss applications of our transducer-based LCU algorithm for implementing sparse matrix functions, including the construction of sparse matrix block-encoding, as well as solving sparse Hamiltonian simulation and linear systems of equations problems.

Let $A \in \mathbb{C}^{N \times N}$ be a Hermitian matrix with at most d non-zero entries per row or column. We assume access to its standard black-box unitary oracles which compute the positions and values of non-zero entries. Specifically,

$$O_F |i, \ell\rangle = |i, f(i, \ell)\rangle, \quad i \in [N], \ell \in [d] \quad (195)$$

$$O_A |i, k, z\rangle = |i, k, z \oplus A_{ik}\rangle, \quad i, k \in [N], \quad (196)$$

where $f(i, \ell)$ is the column index of the ℓ -th non-zero entry in the i -th row, $z \in \{0, 1\}^b$ is a b -bit string, and A_{ik} denotes the matrix entry at i -th row and k -th column. These two oracles O_F and O_A will serve as the “primitive oracles” in Assumption 2.1. We denote $\|A\|$ as the spectral norm of A , $\|A\|_{\max} = \max_{i,k} |A_{ik}|$, and $\|A\|_{1 \rightarrow 2} := \max_k (\sum_i |A_{ik}|^2)^{1/2}$.

5.1 Sparse matrix block-encoding

Now we discuss how to efficiently construct a block-encoding of a sparse matrix. The approach is based on a decomposition $A = \sum_{j=0}^m A_j$ modified from [16], which splits the original sparse matrix into several according to the entries' magnitudes. Then we can construct the block-encoding of each A_j using the standard sparse matrix block-encoding with amplitude amplification in [16], and apply our transducer-based LCU to construct A .

5.1.1 A review of the standard sparse block-encoding with amplification

We first state the standard approach for block encoding a sparse matrix H with amplitude amplification in [16]. The algorithm first constructs two sets of mutually orthogonal quantum states $\{\chi_k\}$, $\{\bar{\chi}_j\}$ such that their overlap $\langle \bar{\chi}_j | \chi_k \rangle = H_{jk} / (d \|H\|_{\max})$. Then H is block-encoded by the product of controlled-state preparation unitary $U = U_{\text{row}}^\dagger U_{\text{col}}$, where $U_{\text{col}}|k\rangle_s|0\rangle_a = |\chi_k\rangle$, $U_{\text{row}}|j\rangle_s|0\rangle_a = |\bar{\chi}_j\rangle$. Both $U_{\text{row}}^\dagger$ and U_{col} can be constructed using $\mathcal{O}(1)$ queries. The normalization factor $d\|H\|_{\max}$ is induced by the normalization factor $\sqrt{\|H\|_{\max}}$ in $U_{\text{row}}^\dagger$ and U_{col} , due to the controlled rotation in constructing them, which require each non-zero entry to be shrunk smaller than 1. However, such a normalization factor is not optimal as the largest possible normalization factor for $U_{\text{row}}^\dagger$ and U_{col} is $\sqrt{\|H\|_1}$, so the algorithm applies the QSVT-based uniform singular value amplification technique to boost the normalization factor of $U_{\text{row}}^\dagger$ and U_{col} to $\sqrt{\Lambda_1}$ for any $\Lambda_1 \geq \|H\|_1$. This process requires $\mathcal{O}(\sqrt{d\|H\|_{\max}/\Lambda_1} \log(1/\epsilon))$ queries, leading to a block-encoding of approximately H/Λ_1 .

We formally summarize the complexity of this procedure in the next lemma, which is from Theorem 10 of [16].

Lemma 5.1 (Sparse block-encoding by amplitude multiplication). *Let $H \in \mathbb{C}^{N \times N}$ be Hermitian and d -sparse. Then, for any $0 < \delta < 1$ and $\Lambda_1 \geq \|H\|_1$, we can construct a block-encoding of a Hermitian matrix $\tilde{H}/\alpha$ with $\alpha = \Theta(\Lambda_1)$ and $\|\tilde{H} - H\| = \mathcal{O}(\|H\|\delta)$, using*

$$\mathcal{O}\left(\sqrt{\frac{d\|H\|_{\max}}{\Lambda_1}} \log\left(\frac{1}{\delta}\right)\right) \quad (197)$$

queries to the sparse matrix oracles.

5.1.2 Matrix decomposition

Now we decompose the matrix A according to the following rule. Let

$$\mu_0 = \frac{\|A\|_{1 \rightarrow 2}}{\sqrt{d}}, \quad \mu_j = 2^j \mu_0 = \frac{2^j \|A\|_{1 \rightarrow 2}}{\sqrt{d}}, \quad (198)$$

and m be the smallest index such that $\mu_m \geq \|A\|_{\max}$. It is clear that

$$m = \mathcal{O}(\log(\sqrt{d}\|A\|_{\max}/\|A\|_{1 \rightarrow 2})). \quad (199)$$

Define the matrix A_j to be

$$(A_0)_{ik} = A_{ik} \mathbf{1}\{|A_{ik}| \leq \mu_0\}, \quad (200)$$

$$(A_j)_{ik} = A_{ik} \mathbf{1}\{\mu_{j-1} < |A_{ik}| \leq \mu_j\}, \quad 1 \leq j \leq m, \quad (201)$$

i.e., A_j consists of A 's non-zero entries between μ_{j-1} and μ_j . Thus $A = \sum_{j=0}^m A_j$.

To apply our transducer-based LCU, we estimate relative quantities in the next result.

Lemma 5.2. Let $A = \sum_{j=0}^m A_j$ where each A_j is defined through Eqs. (200) and (201). Then, for any $0 < \epsilon < 1$, we can implement block-encodings U_j of $\tilde{A}_j/\alpha_j$ with query complexity C_j , such that

$$\lambda = \sum_{j=0}^m \alpha_j = \mathcal{O}\left(\sqrt{d}\|A\|_{1 \rightarrow 2}\right), \quad (202)$$

$$\bar{C} = \frac{\sum_{j=0}^m \alpha_j C_j}{\lambda} = \mathcal{O}\left(\log\left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}}\right) \log\left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon}\right)\right), \quad (203)$$

$$C_{\max} = \max_j C_j = \mathcal{O}\left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}} \log\left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon}\right)\right), \quad (204)$$

and

$$\|\tilde{A} - A\| \leq \mathcal{O}(\epsilon), \quad \tilde{A} := \sum_{j=0}^m \tilde{A}_j. \quad (205)$$

Proof. We first show that

$$\|A_0\|_1 \leq \sqrt{d}\|A\|_{1 \rightarrow 2}, \quad \|A_j\|_1 \leq \frac{\|A\|_{1 \rightarrow 2}^2}{\mu_{j-1}}, \quad \forall 1 \leq j \leq m. \quad (206)$$

For A_0 , as all the entries have absolute values smaller than μ_0 , and there are at most d non-zero entries per column, we have

$$\|A_0\|_1 \leq d\mu_0 = \sqrt{d}\|A\|_{1 \rightarrow 2}. \quad (207)$$

For $j \geq 1$, for every non-zero entry $(A_j)_{ik}$, we first use $|(A_j)_{ik}| > \mu_{j-1}$ to obtain $|(A_j)_{ik}| \leq \frac{|(A_j)_{ik}|^2}{\mu_{j-1}}$, and then

$$\|A_j\|_1 = \max_i \sum_k |(A_j)_{ik}| \leq \frac{\max_i \sum_k |(A_j)_{ik}|^2}{\mu_{j-1}} \leq \frac{\|A\|_{1 \rightarrow 2}^2}{\mu_{j-1}}. \quad (208)$$

We have proved Eq. (206).

According to Eq. (206), we can choose the normalization factor of $\tilde{A}_j$ as

$$\alpha_0 = \sqrt{d}\|A\|_{1 \rightarrow 2}, \quad \alpha_j = \frac{\|A\|_{1 \rightarrow 2}^2}{\mu_{j-1}} = \frac{\sqrt{d}\|A\|_{1 \rightarrow 2}}{2^{j-1}}, \quad \forall 1 \leq j \leq m. \quad (209)$$

Taking the sum, we first have

$$\lambda = \sum_{j=0}^m \alpha_j = \Theta\left(\sqrt{d}\|A\|_{1 \rightarrow 2}\right). \quad (210)$$

For any $\delta > 0$ to be specified later, using Lemma 5.1, we can construct block-encodings U_j of $\tilde{A}_j/\alpha_j$ with query complexity C_j , such that

$$\|\tilde{A}_j - A_j\| \leq \mathcal{O}(\|A_j\|\delta), \quad C_j = \mathcal{O}\left(\sqrt{\frac{d\mu_j}{\alpha_j}} \log\left(\frac{1}{\delta}\right)\right). \quad (211)$$

For error, we simply take the sum with respect to j and obtain

$$\|\tilde{A} - A\| \leq \sum_{j=0}^m \|\tilde{A}_j - A_j\| = \mathcal{O}(\lambda\delta). \quad (212)$$

So we can choose $\delta = \epsilon/\lambda$ to bound the overall error by $\mathcal{O}(\epsilon)$, and the corresponding query complexity becomes

$$C_j = \mathcal{O} \left(\sqrt{\frac{d\mu_j}{\alpha_j}} \log \left(\frac{\lambda}{\epsilon} \right) \right) = \mathcal{O} \left(\sqrt{\frac{d\mu_j}{\alpha_j}} \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \right). \quad (213)$$

Then, using $\mu_j = 2\mu_{j-1} = \frac{2\|A\|_{1 \rightarrow 2}^2}{\alpha_j}$ for $j \geq 1$ and $\mu_0 = \frac{\|A\|_{1 \rightarrow 2}^2}{\alpha_0}$, we can compute

$$\bar{C} = \frac{\sum_{j=0}^m \alpha_j C_j}{\lambda} = \mathcal{O} \left(\frac{1}{\lambda} \sum_{j=0}^m \alpha_j \sqrt{\frac{d\mu_j}{\alpha_j}} \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \right) \quad (214)$$

$$= \mathcal{O} \left(m \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \right) = \mathcal{O} \left(\log \left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}} \right) \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \right). \quad (215)$$

To bound the maximum complexity, notice that $\sqrt{d\mu_0/\alpha_0} = \mathcal{O}(1)$ and $\sqrt{d\mu_j/\alpha_j} = \mathcal{O}(2^j)$, we have

$$C_{\max} = \max_j C_j = \mathcal{O} \left(2^{m-1} \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \right) = \mathcal{O} \left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}} \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \right), \quad (216)$$

where the last estimate is from $\mu_{m-1} < \|A\|_{\max}$ according to the definition of m . $\square$

5.1.3 Complexity

The complexity of applying transducer-based LCU to implement the above decomposition directly follows from Theorem 4.2 and Lemma 5.2.

Theorem 5.3 (Sparse block-encoding via transducer-based LCU). *Let A be a d -sparse Hermitian matrix with a known upper bound $a \geq \|A\|$ and known $\|A\|_{\max}$, $\|A\|_{1 \rightarrow 2}$. Then, for any $0 < \epsilon \leq \mathcal{O}(a)$, we can construct a block-encoding of A'/α with $\alpha = \Theta(a)$ and $\|A' - A\| \leq \mathcal{O}(\epsilon)$, using*

$$\mathcal{O} \left(\left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}} + \frac{\sqrt{d}\|A\|_{1 \rightarrow 2}}{a} \log \left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}} \right) \log \left(\frac{a}{\epsilon} \right) \right) \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \log \left(\frac{a}{\epsilon} \right) \right) \quad (217)$$

queries to the sparse oracle access to A .

Proof. By Lemma 5.2, we have block-encodings U_j of $\tilde{A}_j/\alpha_j$ such that $\|\tilde{A} - A\| \leq \mathcal{O}(\epsilon)$, $\tilde{A} = \sum_{j=0}^m \tilde{A}_j$. Since $\|A\| \leq a$, we can use $\beta = \Theta(a)$ as an upper bound of $\|\tilde{A}\|$. Then, Theorem 4.2 gives a block-encoding of $A'/(16\beta)$ such that $\|A'/(16\beta) - \tilde{A}/(16\beta)\| \leq \mathcal{O}(\epsilon/\beta)$, which in turn implies $\|A' - A\| \leq \mathcal{O}(\epsilon)$.

According to Theorem 4.2 and Lemma 5.2, query complexity of this algorithm is

$$\mathcal{O} \left(C_{\max} \log \left(\frac{a}{\epsilon} \right) + \frac{\lambda}{a} \bar{C} \left(\log \left(\frac{a}{\epsilon} \right) \right)^2 \right) \quad (218)$$

$$= \mathcal{O} \left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}} \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \log \left(\frac{a}{\epsilon} \right) \right) \quad (219)$$

$$+ \frac{\sqrt{d}\|A\|_{1 \rightarrow 2}}{a} \log \left(\frac{\sqrt{d}\|A\|_{\max}}{\|A\|_{1 \rightarrow 2}} \right) \log \left(\frac{d\|A\|_{1 \rightarrow 2}}{\epsilon} \right) \left(\log \left(\frac{a}{\epsilon} \right) \right)^2. \quad (220)$$

$\square$

A particular useful case is when the matrix A has spectral norm bounded by 1. Then we may choose $a = 1$, and use the inequalities $\|A\|_{\max}/\|A\|_{1 \rightarrow 2} \leq 1$ and $\|A\|_{1 \rightarrow 2} \leq \|A\| \leq 1$ to simplify the expression of the query complexity.

Corollary 5.4 (Sparse block-encoding via transducer-based LCU for normalized matrix). *Let A be a d -sparse Hermitian matrix with $\|A\| \leq 1$. Then, for any $0 < \epsilon \leq \mathcal{O}(1)$, we can construct a block-encoding of A'/α with $\alpha = \mathcal{O}(1)$ and $\|A' - A\| \leq \mathcal{O}(\epsilon)$, using*

$$\mathcal{O} \left(\sqrt{d} \log(d) \log \left(\frac{d}{\epsilon} \right) \left(\log \left(\frac{1}{\epsilon} \right) \right)^2 \right) \quad (221)$$

queries to the sparse oracle access to A .

5.2 Sparse Hamiltonian simulation

For an evolution time $T > 0$ and a target error $\epsilon > 0$, the goal of the Hamiltonian simulation problem for a Hamiltonian A is to approximate e^{-iAT} to error ϵ . We assume $\|A\| \leq 1$ as we can rescale the dynamics to absorb the spectral norm into the evolution time.

QSVT algorithm can solve the Hamiltonian simulation problem with $\mathcal{O}(\alpha T + \log(1/\epsilon))$ queries to the block-encoding of A/α . We can directly apply our Corollary 5.4 to obtain such a block-encoding. A nuance is that our block-encoding is not exact. By $\|e^{-iAT} - e^{-iA'T}\| \leq T\|A - A'\|$, it suffices to choose the block-encoding error $\mathcal{O}(\epsilon/T)$ to bound the overall simulation error still by $\mathcal{O}(\epsilon)$. Summarizing these discussions, we have the following result.

Theorem 5.5. *Let A be a d -sparse Hamiltonian with $\|A\| \leq 1$. Then, for an evolution time T and a target error ϵ , there exists a quantum algorithm which implements a block-encoding of V such that $\|V - e^{-iAT}\| \leq \mathcal{O}(\epsilon)$, using*

$$\mathcal{O} \left(\left(T + \log \left(\frac{1}{\epsilon} \right) \right) \sqrt{d} \log(d) \log \left(\frac{dT}{\epsilon} \right) \left(\log \left(\frac{T}{\epsilon} \right) \right)^2 \right) \quad (222)$$

queries to the sparse oracle access of A .

5.3 Sparse linear system algorithm

The quantum linear system problem considers

$$Ax = |b\rangle, \quad (223)$$

and the goal of a quantum linear system algorithm is to prepare a quantum state $|\tilde{x}\rangle$ such that

$$\| |\tilde{x}\rangle - |x\rangle \| \leq \epsilon, \quad (224)$$

where $|x\rangle = x/\|x\|$ is the normalized solution of the linear system. Since the goal is the normalized solution, we assume the right hand side vector $|b\rangle$ to be normalized without loss of generality. For the coefficient matrix, we also assume $\|A\| = 1$ and denote its condition number by $\kappa = \|A\| \|A^{-1}\|$. We further restrict A to be a Hermitian matrix, and general non-Hermitian coefficient matrix can be reduced to this Hermitian case by the standard dilation trick with the help of one extra qubit [30]. Query complexity of a quantum linear system algorithm is counted as the number of queries to O_F and O_A , and the state preparation oracle $O_b : |0\rangle \mapsto |b\rangle$.

There have been several existing works which solve the quantum linear system problem using the matrix block-encoding input model. Our sparse matrix block-encoding technique can be naturally combined with those algorithms to improve the dependence on the sparsity.

For example, with the block-encoding of A'/α , the discrete adiabatic algorithm [27] can solve the linear system $A'x' = |b\rangle$ to error in the normalized state at most ϵ , using $\mathcal{O}(\alpha \|A'^{-1}\| \log(1/\epsilon))$

queries to O_F, O_A and O_b . Furthermore, for any $\epsilon > 0$, we can ensure $\|x/\|x\| - x'/\|x'\|\| \leq \mathcal{O}(\epsilon)$, by choosing the error in the block-encoding to be ⁶ $\mathcal{O}(\epsilon/\|A^{-1}\|)$. Putting this with Corollary 5.4 together, the query complexity becomes $\mathcal{O}(\sqrt{d}\kappa \text{polylog}(d\kappa/\epsilon))$.

Alternatively, we may apply the tunable variable time amplitude amplification (tunable VTAA) [19] to trade off between matrix query complexity and state preparation cost. For $A'x' = b$, it requires $\mathcal{O}(\alpha\kappa \log(1/p') \log(\log(1/p')/\epsilon))$ queries to the block-encoding of A'/α , and $\mathcal{O}(1/\sqrt{p'})$ to O_b , where $1/\sqrt{p'} = \|A'^{-1}\|/\|A'^{-1}|b\rangle\|$. Using $\|A'^{-1}\|/\|A'^{-1}|b\rangle\| \sim \|A^{-1}\|/\|A^{-1}|b\rangle\|$, $\|A'^{-1}\| \sim \|A^{-1}\|$, and choosing a similar block-encoding accuracy as in the discrete adiabatic approach, we can also solve $Ax = |b\rangle$ with error at most $\mathcal{O}(\epsilon)$.

We summarize these two approaches in the following result.

Theorem 5.6. *Let A be a d -sparse Hermitian matrix with $\|A\| = 1$ and condition number κ , and $|b\rangle$ be a normalized vector. Then, for any $\epsilon \leq \mathcal{O}(1/\kappa)$:*

1. *The discrete adiabatic algorithm can prepare a quantum state $|\tilde{x}\rangle$ such that $\| |\tilde{x}\rangle - \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|} \| \leq \mathcal{O}(\epsilon)$, using*

$$\mathcal{O}\left(\kappa\sqrt{d}\log(d)\log\left(\frac{d\kappa}{\epsilon}\right)\left(\log\left(\frac{\kappa}{\epsilon}\right)\right)^2\log\left(\frac{1}{\epsilon}\right)\right) \quad (225)$$

queries to the sparse oracle access to A and the state preparation oracle of $|b\rangle$.

2. *Alternatively, the tunable VTAA approach can achieve the same goal, using*

$$\mathcal{O}\left(\kappa\sqrt{d}\log(d)\log\left(\frac{d\kappa}{\epsilon}\right)\left(\log\left(\frac{\kappa}{\epsilon}\right)\right)^2\log\left(\frac{1}{p}\right)\log\left(\frac{\log(1/p)}{\epsilon}\right)\right) \quad (226)$$

queries to the sparse oracle access to A , and

$$\mathcal{O}\left(\frac{1}{\sqrt{p}}\right) \quad (227)$$

queries to the state preparation oracle of $|b\rangle$. Here $1/\sqrt{p} = \|A^{-1}\|/\|A^{-1}|b\rangle\|$, and we assume an estimate of $\|A^{-1}|b\rangle\|$ up to constant multiplicative error is known a priori.

6 Conclusion and outlook

In this work, we developed a transducer-based LCU framework that replaces the multiplicative worst-case cost dependence of standard LCU by a weighted-average dependence under a shared primitive-access assumption. We also developed resolvent-based tools for composing and implementing transducers, and applied the resulting framework to sparse matrices, obtaining near-optimal query complexities for sparse Hamiltonian simulation and quantum linear systems up to polylogarithmic factors.

A natural first direction is to extend the ideas developed here beyond LCU to QSVT. QSVT is a considerably more general framework for quantum matrix algorithms, and it would be interesting to understand whether transducer techniques can similarly exploit nonuniform costs within QSVT constructions and lead to refined complexity bounds. Such an extension could potentially provide a systematic route to further applications.

A second direction is to further develop the transducer framework itself. Our analysis identifies the resolvent norm $K(S)$ as a useful complexity measure alongside the catalyst complexity. It would be interesting to clarify the relation between these two measures, establish sharper bounds for the resolvent norm under transducer composition, and determine whether the $\mathcal{O}(K(S)\log(1/\epsilon))$ complexity of our finite-reuse implementation can be further improved.

⁶This claim can be proved as follows. Let $\|A' - A\| = \delta$ and write $A' = A + E$, $u = x/\|x\|$, $v = x'/\|x'\|$, and $r = \|x\|/\|x'\|$. Since $x = x' + A^{-1}Ex'$ and $\|ru - v\| \leq \delta\|A^{-1}\|$ (we force $\delta\|A^{-1}\| \leq 1/2$), we can bound $\text{Re}\langle u|v\rangle \geq \frac{r^2+1-(\delta\|A^{-1}\|)^2}{2r} \geq \sqrt{1-(\delta\|A^{-1}\|)^2}$. Then the claimed bound follows from $\|u-v\| = \sqrt{2-2\text{Re}\langle u|v\rangle} = \mathcal{O}(\delta\|A^{-1}\|)$.

Acknowledgements

DA, DD, and CZ acknowledge funding from Quantum Science and Technology - National Science and Technology Major Project via Project 2024ZD0301900, and the support by The Fundamental Research Funds for the Central Universities, Peking University. CS was supported by the National Key Research Project of China under Grant No. 2025YFA1017200. YZ was supported by the European Research Council (ERC) via the Starting Grant QuanThermal (101222179).

AI use statement

This work involved extensive interaction between the authors and a large language model (LLM). The authors initially aimed to develop a quantum algorithm for sparse linear systems with optimal dependence on all relevant parameters. During this stage, the LLM was used for brainstorming and suggested using the original transducer framework [21], which could achieve the desired sparsity dependence but not the target precision dependence. After the appearance of [23], the authors recognized that an improved implementation of transducers could yield better precision dependence and discussed this idea with LLM. The LLM then assisted in developing an early version of sparse linear system solver with near-optimal dependence on all parameters.

While refining that early version, the authors realized that the techniques could be extended to the broader task of LCU. The authors consequently shifted the motivation and focus of the work to developing an improved LCU primitive through the transducer framework, with sparse matrix functions serving as applications. The authors formulated the central idea and key steps of the transducer-based LCU algorithm, while the LLM assisted with the analysis and the development of technical results. Building on these discussions, the authors further developed and refined the analysis and proofs.

The authors wrote the initial manuscript, and used the LLM to refine the presentation of selected sections. The authors take full responsibility for all content presented in this work.

References

- [1] Andrew M Childs, Robin Kothari, and Rolando D Somma. Quantum algorithm for systems of linear equations with exponentially improved dependence on precision. *SIAM Journal on Computing*, 46(6):1920–1950, 2017.
- [2] Dominic W Berry, Andrew M Childs, Richard Cleve, Robin Kothari, and Rolando D Somma. Simulating Hamiltonian dynamics with a truncated Taylor series. *Physical Review Letters*, 114(9):090502, 2015.
- [3] Dominic W Berry, Andrew M Childs, Richard Cleve, Robin Kothari, and Rolando D Somma. Exponential improvement in precision for simulating sparse Hamiltonians. In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing*, pages 283–292, 2014.
- [4] Dong An, Jin-Peng Liu, and Lin Lin. Linear combination of Hamiltonian simulation for nonunitary dynamics with optimal state preparation cost. *Physical Review Letters*, 131(15):150603, 2023.
- [5] Dong An, Andrew M. Childs, and Lin Lin. Quantum algorithm for linear non-unitary dynamics with near-optimal dependence on all parameters. *Communications in Mathematical Physics*, 407(1):19, 2026.

-
- [6] Guang Hao Low and Rolando D Somma. Optimal quantum simulation of linear non-unitary dynamics. *arXiv preprint arXiv:2508.19238*, 2025.
 - [7] Simon Apers, Shantanav Chakraborty, Leonardo Novo, and Jérémie Roland. Quadratic speedup for spatial search by continuous-time quantum walk. *Physical Review Letters*, 129(16):160502, 2022.
 - [8] Andris Ambainis, András Gilyén, Stacey Jeffery, and Martins Kokainis. Quadratic speedup for finding marked vertices by quantum walks. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 412–424, 2020.
 - [9] Trevor Keen, Eugene Dumitrescu, and Yan Wang. Quantum algorithms for ground-state preparation and Green’s function calculation. *arXiv:2112.05731*, 2021.
 - [10] Yimin Ge, Jordi Tura, and J Ignacio Cirac. Faster ground state preparation and high-precision ground energy estimation with fewer qubits. *Journal of Mathematical Physics*, 60(2):022202, 2019.
 - [11] Shantanav Chakraborty, Soumyabrata Hazra, Tongyang Li, Changpeng Shao, Xinzhaoh Wang, and Yuxin Zhang. Quantum singular value transformation without block encodings: Near-optimal complexity with minimal ancilla. *arXiv:2504.02385*, 2025.
 - [12] Guang Hao Low and Yuan Su. Quantum eigenvalue processing. *SIAM Journal on Computing*, 55(1):135–215, 2026.
 - [13] Dong An, Andrew M Childs, Lin Lin, and Lexing Ying. Laplace transform based quantum eigenvalue transformation via linear combination of hamiltonian simulation. *arXiv preprint arXiv:2411.04010*, 2024.
 - [14] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 193–204, 2019. *arXiv:1806.01838*.
 - [15] Andrew M. Childs and Nathan Wiebe. Hamiltonian simulation using linear combinations of unitary operations. *Quantum Information and Computation*, 12(11–12), 2012.
 - [16] Guang Hao Low. Hamiltonian simulation with nearly optimal dependence on spectral norm. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 491–502, 2019.
 - [17] Hitomi Mori, Yuta Kikuchi, Marcello Benedetti, and Matthias Rosenkranz. Sparsity-dependent complexity lower bound of quantum linear system solvers. *Quantum Science and Technology*, 11(3):035063, 2026.
 - [18] Dhruvil Patel. A near-optimal joint lower bound for sparse quantum linear system solvers. *arXiv:2609.22445*, 2026.
 - [19] Guang Hao Low and Yuan Su. Quantum linear system algorithm with optimal queries to initial state preparation. *Quantum*, 10:2041, 2026.
 - [20] Ashley Montanaro and Changpeng Shao. Quantum and classical query complexities of functions of matrices. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, pages 573–584. Association for Computing Machinery, 2024.
 - [21] Aleksandrs Belovs, Stacey Jeffery, and Duyal Yolcu. Taming quantum time complexity. *Quantum*, 8:1444, August 2024.
-

-
- [22] Simon Apers, Jérémie Roland, and Yuxin Zhang. Elfs, transducers and quantum walks. *arXiv:2605.30013*, 2026.
 - [23] Boyang Chen, Minbo Gao, Xinzhao Wang, and Shuo Zhou. Time-dependent Hamiltonian simulation with optimal query complexity. *arXiv:2608.06094*, 2026.
 - [24] Boyang Chen, Minbo Gao, Zhengfeng Ji, Tongyang Li, Xinzhao Wang, and Shuo Zhou. Gate-efficient implementation of the query-optimal time-dependent Hamiltonian simulation. *arXiv:2608.30629*, 2026.
 - [25] Chunhao Wang and Christopher Ye. Query-optimal quantum simulation of Lindblad evolution. *arXiv:2609.17490*, 2026.
 - [26] Boyang Chen, Minbo Gao, Xinzhao Wang, and Shuo Zhou. Query-optimal and gate-efficient lindbladian simulation, 2026.
 - [27] Pedro C.S. Costa, Dong An, Yuval R. Sanders, Yuan Su, Ryan Babbush, and Dominic W. Berry. Optimal scaling quantum linear-systems solver via discrete adiabatic theorem. *PRX Quantum*, 3:040303, 2022.
 - [28] Guang Hao Low and Isaac L. Chuang. Optimal hamiltonian simulation by quantum signal processing. *Physical Review Letters*, 118(1):010501, January 2017.
 - [29] Guang Hao Low and Isaac L Chuang. Hamiltonian simulation by qubitization. *Quantum*, 3:163, 2019.
 - [30] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15):150502, 2009.
 - [31] Andris Ambainis. Variable time amplitude amplification and quantum algorithms for linear algebra problems. In *29th Symposium on Theoretical Aspects of Computer Science*, volume 14, pages 636–647, 2012.
 - [32] Shantanav Chakraborty, András Gilyén, and Stacey Jeffery. The power of block-encoded matrix powers: Improved regression techniques via faster Hamiltonian simulation. In *Proceedings of the 46th International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 33:1–33:14, 2019. *arXiv:1804.01973*.
 - [33] Yiğit Subaşı, Rolando D. Somma, and Davide Orsucci. Quantum algorithms for systems of linear equations inspired by adiabatic quantum computing. *Physical Review Letters*, 122:060504, 2019.
 - [34] Alexander M Dalzell. A shortcut to an optimal quantum linear system solver. *arXiv:2406.12086*, 2024.
 - [35] Jianqiang Li. A new quantum linear system algorithm beyond the condition number and its application to solving multivariate polynomial systems. *arXiv:2510.05588*, 2025.
 - [36] Alexander M Dalzell, Jianqiang Li, and Yuan Su. Faster quantum linear system solver beyond the condition number. *arXiv:2607.07691*, 2026.
 - [37] Stacey Jeffery and Freek Witteveen. Optimal quantum algorithm for ground-state energy estimation with a guiding state. *arXiv:2608.24494*, 2026.
 - [38] Carlos Bravo-Prieto, Aram W. Harrow, and Robin Kothari. An optimal quantum linear systems algorithm. *arXiv:2609.35660*, 2026.
-

-
- [39] Mikko Mottonen, Juha J Vartiainen, Ville Bergholm, and Martti M Salomaa. Transformation of quantum states using uniformly controlled rotations. *quant-ph/0407010*, 2004.
 - [40] Lov Grover and Terry Rudolph. Creating superpositions that correspond to efficiently integrable probability distributions. *quant-ph/0208112*, 2002.
 - [41] Fuzhen Zhang, editor. *The Schur Complement and Its Applications*, volume 4 of *Numerical Methods and Algorithms*. Springer, New York, 2005.
 - [42] Douglas E. Crabtree and Emilie V. Haynsworth. An identity for the Schur complement of a matrix. *Proceedings of the American Mathematical Society*, 22(2):364–366, 1969.
 - [43] L. F. Richardson. The approximate arithmetical solution by finite differences of physical problems involving differential equations, with an application to the stresses in a masonry dam. *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, 210:307–357, 1911.
 - [44] Roland Bulirsch and Josef Stoer. Numerical treatment of ordinary differential equations by extrapolation methods. *Numerische Mathematik*, 8:1–13, 1966.
 - [45] Guang Hao Low, Vadym Kliuchnikov, and Nathan Wiebe. Well-conditioned multiproduct Hamiltonian simulation. *arXiv:1907.11679*, 2019.
 - [46] Junaid Aftab, Dong An, and Konstantina Trivisa. Multi-product Hamiltonian simulation with explicit commutator scaling. *arXiv:2403.08922*, 2024.
 - [47] James D Watson and Jacob Watkins. Exponentially reduced circuit depths using trotter error mitigation. *PRX quantum*, 6(3):030325, 2025.
 - [48] Di Fang and Justin Park. Circuit depth reduction of one-ancilla quantum differential equation solver via extrapolation. *arXiv preprint arXiv:2607.07389*, 2026.
 - [49] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. Error mitigation for short-depth quantum circuits. *Physical Review Letters*, 119:180509, Nov 2017.
 - [50] Ying Li and Simon C. Benjamin. Efficient variational quantum simulator incorporating active error minimization. *Physical Review X*, 7:021050, Jun 2017.
 - [51] Dominic W Berry, Andrew M Childs, Richard Cleve, Robin Kothari, and Rolando D Somma. Simulating Hamiltonian dynamics with a truncated Taylor series. *Physical Review Letters*, 114(9):090502, 2015.

A Polynomial approximation of the inverse Cayley transformation and amplification

We use the following result on polynomial approximations of analytic functions.

Lemma A.1 (Corollary 66 of [14]). *Let $x_0 \in [-1, 1]$, $r \in (0, 2]$, $\eta \in (0, r]$, and let $f: [x_0 - r - \eta, x_0 + r + \eta] \rightarrow \mathbb{C}$ be such that $f(x_0 + x) = \sum_{\ell=0}^{\infty} a_{\ell} x^{\ell}$ for all $x \in [-r - \eta, r + \eta]$. Suppose $B > 0$ is such that $\sum_{\ell=0}^{\infty} (r + \eta)^{\ell} |a_{\ell}| \leq B$, and let $\epsilon \in (0, \frac{1}{2B}]$. Then there is an efficiently computable polynomial $\tilde{P} \in \mathbb{C}[x]$ of degree $\mathcal{O}(\frac{1}{\eta} \log \frac{B}{\epsilon})$ such that*

$$\|f - \tilde{P}\|_{[x_0-r, x_0+r]} \leq \epsilon \quad \text{and} \quad \|\tilde{P}\|_{[-1,1]} \leq \epsilon + B. \quad (228)$$

Lemma A.2. *For any $0 < \rho \leq 1/4$ and $0 < \delta < 1/2$, there exists a real odd polynomial $P(x)$ such that*

$$|P(x)| \leq \frac{1}{2}, \quad \forall x \in [-1, 1], \quad (229)$$

$$\left| P\left(\frac{2b}{1+b^2}\right) - \frac{b}{16\rho} \right| \leq \delta, \quad \forall b \in [-\rho, \rho], \quad (230)$$

$$\deg(P(x)) = \mathcal{O}\left(\frac{1}{\rho} \log\left(\frac{1}{\delta}\right)\right). \quad (231)$$

Proof. Let $g(x) := \frac{x}{1+\sqrt{1-x^2}}$. For $|b| \leq 1$, a direct computation gives $g\left(\frac{2b}{1+b^2}\right) = b$, and $\left|\frac{2b}{1+b^2}\right| \leq 2\rho$ whenever $|b| \leq \rho$. Hence it suffices to approximate $f(x) := g(x)/(16\rho)$ on $[-2\rho, 2\rho]$. Since $g(x) = (1 - \sqrt{1-x^2})/x = \sum_{k \geq 1} c_k x^{2k-1}$ with $c_k \geq 0$ for $|x| \leq 1$, the coefficients a_ℓ of f satisfy

$$\sum_{\ell} (4\rho)^\ell |a_\ell| = \frac{g(4\rho)}{16\rho} \leq \frac{1}{4}. \quad (232)$$

Let $\epsilon_0 := \min\{\delta, 1/16\}$. Applying Lemma A.1 with $x_0 = 0$, $r = \eta = 2\rho$, $B = 1/4$ and $\epsilon = \epsilon_0$ yields a polynomial $\tilde{P}$ of degree $\mathcal{O}(\rho^{-1} \log(1/\delta))$ such that

$$\|f - \tilde{P}\|_{[-2\rho, 2\rho]} \leq \epsilon_0 \quad \text{and} \quad \|\tilde{P}\|_{[-1, 1]} \leq \epsilon_0 + \frac{1}{4} \leq \frac{5}{16}. \quad (233)$$

Since f is real and odd, the odd part of the real part of $\tilde{P}$, i.e. $P(x) := \frac{1}{2}(\operatorname{Re} \tilde{P}(x) - \operatorname{Re} \tilde{P}(-x))$, satisfies the same bounds. Specifically, we have

$$\|P\|_{[-1, 1]} < \frac{1}{2} \quad \text{and} \quad \|f - P\|_{[-2\rho, 2\rho]} \leq \epsilon_0 \leq \delta, \quad (234)$$

which completes the proof. $\square$

B A simple explanation of the construction in Lemma 3.2

The construction in Lemma 3.2 admits a natural interpretation in terms of a random walk on a directed cycle. We illustrate this for $L = 3$. Consider

$$S = \begin{pmatrix} 0 & 0 & V_2 \\ V_0 & 0 & 0 \\ 0 & V_1 & 0 \end{pmatrix}. \quad (235)$$

This operator, inspired by (33), can be viewed as a one-step walk operator on a directed cycle with vertices $0, 1, 2$, where the transition from vertex j is accompanied by the unitary V_j . Indeed,

$$|0\rangle \otimes |x\rangle \xrightarrow{S} |1\rangle \otimes V_0 |x\rangle \xrightarrow{S} |2\rangle \otimes V_1 V_0 |x\rangle \xrightarrow{S} |0\rangle \otimes V_2 V_1 V_0 |x\rangle. \quad (236)$$

Equivalently, if

$$S \begin{pmatrix} x \\ y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} Vx \\ y_1 \\ y_2 \end{pmatrix}, \quad V = V_2 V_1 V_0, \quad (237)$$

then $y_1 = V_0 x$, $y_2 = V_1 V_0 x$. The same picture extends to the composition of transducers. Let

$$S_i = \begin{pmatrix} D_i & E_i \\ B_i & T_i \end{pmatrix}, \quad (238)$$

and consider the direct sum $S_0 \oplus S_1 \oplus S_2$. Grouping the public variables first and the private variables second, we have

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ y_0 \\ y_1 \\ y_2 \end{pmatrix} \xrightarrow{S_0 \oplus S_1 \oplus S_2} \begin{pmatrix} D_0 x_0 + E_0 y_0 \\ D_1 x_1 + E_1 y_1 \\ D_2 x_2 + E_2 y_2 \\ B_0 x_0 + T_0 y_0 \\ B_1 x_1 + T_1 y_1 \\ B_2 x_2 + T_2 y_2 \end{pmatrix}. \quad (239)$$

To reproduce the directed cycle $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$, we cyclically shift only the public outputs while leaving the private outputs at their original vertices. This gives

$$\begin{pmatrix} D_2 x_2 + E_2 y_2 \\ D_0 x_0 + E_0 y_0 \\ D_1 x_1 + E_1 y_1 \\ B_0 x_0 + T_0 y_0 \\ B_1 x_1 + T_1 y_1 \\ B_2 x_2 + T_2 y_2 \end{pmatrix} = \begin{pmatrix} 0 & 0 & D_2 & 0 & 0 & E_2 \\ D_0 & 0 & 0 & E_0 & 0 & 0 \\ 0 & D_1 & 0 & 0 & E_1 & 0 \\ B_0 & 0 & 0 & T_0 & 0 & 0 \\ 0 & B_1 & 0 & 0 & T_1 & 0 \\ 0 & 0 & B_2 & 0 & 0 & T_2 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ y_0 \\ y_1 \\ y_2 \end{pmatrix}. \quad (240)$$

This is exactly the construction in (47) for $L = 3$.

This walk interpretation also helps explain the bound on $K(S)$. Consider

$$(I - T) \begin{pmatrix} h_1 \\ h_2 \\ v_0 \\ v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ b_0 \\ b_1 \\ b_2 \end{pmatrix}. \quad (241)$$

Writing

$$R_j = (I - T_j)^{-1}, \quad \Gamma_j = R_j B_j, \quad (242)$$

the local private equations give

$$\begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} R_0 & 0 & 0 \\ 0 & R_1 & 0 \\ 0 & 0 & R_2 \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ b_2 \end{pmatrix} + \begin{pmatrix} 0 \\ \Gamma_1 h_1 \\ \Gamma_2 h_2 \end{pmatrix}. \quad (243)$$

which is exactly (65). The first term is purely local and is therefore controlled by

$$\max_j \|R_j\| = \max_j K(S_j). \quad (244)$$

The second term is generated by the intermediate clock variables h_1, h_2 , which collect residuals propagated along the directed walk. To be a little more exact, the second term is generated by the intermediate clock variables h_1, h_2 . These variables themselves contain residuals propagated from earlier vertices. Indeed,

$$h_1 = a_1 + V_0 \Gamma_0^\dagger b_0, \quad h_2 = V_1 h_1 + a_2 + V_1 \Gamma_1^\dagger b_1. \quad (245)$$

Thus a local residual b_j first enters the clock propagation through the map $\Gamma_j^\dagger$, is transported along the directed walk by the unitaries V_j , and is finally lifted back into the local private variables through the maps Γ_i . Since the V_j 's are unitary, the propagation itself does not increase the norm, while the entrance and exit maps are controlled by $\|\Gamma_j\| = \sqrt{w(S_j)}$. More precisely, the map from a pair of residuals (a_j, b_{j-1}) to the corresponding clock source has norm at most

$$\left\| \begin{pmatrix} I & V_{j-1} \Gamma_{j-1}^\dagger \end{pmatrix} \right\| \leq \sqrt{1 + \max_j w(S_j)}, \quad (246)$$

whereas reconstructing the public/private pair $h_j \mapsto \begin{pmatrix} h_j \\ \Gamma_j h_j \end{pmatrix}$ costs another factor of at most $\sqrt{1 + \max_j w(S_j)}$. The residuals can propagate through at most $2 = L - 1$ intermediate clock positions. Consequently, the propagation contribution is bounded by

$$2(1 + \max_j w(S_j)). \quad (247)$$

Thus the bound on $K(S)$ in Lemma 3.2 naturally separates into a local elimination contribution, governed by $\max_j K(S_j)$, and a propagation contribution arising from the accumulation of residuals along the clock path. The former reflects the worst local sensitivity of the individual transducers. Regarding the latter, the factor $L - 1$ comes from propagation through the intermediate clock variables, whereas $w(S_j) = \|\Gamma_j\|^2$ controls the coupling between the clock variables and the corresponding local private spaces. In this sense, the two terms represent two distinct mechanisms: local Schur elimination and global propagation along the clock. This decomposition is essentially tight at the level of scaling: in general, neither the dependence on $\max_j K(S_j)$ nor the linear accumulation in L and $1 + \max_j w(S_j)$ can be removed.

Connections to products of block-encodings. The clock construction in Lemma 3.2 provides a query-efficient method for composing multiple transducers. Here we describe an alternative sequential construction, which is more closely related to the usual composition of block encodings [14]. Although this construction does not necessarily improve the worst-case bound on $K(S)$ compared with the clock construction, its Schur-complement structure reveals a different multiplicative behavior of the resolvent norm.

We start from the case $L = 2$. Let

$$S_j = \begin{pmatrix} D_j & E_j \\ B_j & T_j \end{pmatrix}, \quad j = 0, 1, \quad (248)$$

be transducers of V_j with catalyst maps Γ_j . We embed them into the common space $\mathcal{H}_{pub} \oplus \mathcal{H}_0 \oplus \mathcal{H}_1$ and define

$$\widehat{S}_0 = \begin{pmatrix} D_0 & E_0 & 0 \\ B_0 & T_0 & 0 \\ 0 & 0 & I \end{pmatrix}, \quad \widehat{S}_1 = \begin{pmatrix} D_1 & 0 & E_1 \\ 0 & I & 0 \\ B_1 & 0 & T_1 \end{pmatrix}. \quad (249)$$

The sequential composition

$$S_{\text{seq}} = \widehat{S}_1 \widehat{S}_0 \quad (250)$$

is a unitary operator. One may realize that S_{seq} is indeed the standard way of constructing block-encodings of the product $D_1 D_0$. Define the catalyst map

$$\Gamma_{\text{seq}} x = \begin{pmatrix} \Gamma_0 x \\ \Gamma_1 V_0 x \end{pmatrix}. \quad (251)$$

Using $D_j + E_j \Gamma_j = V_j$, $B_j + T_j \Gamma_j = \Gamma_j$, one directly verifies that

$$S_{\text{seq}} \begin{pmatrix} x \\ \Gamma_{\text{seq}} x \end{pmatrix} = \begin{pmatrix} V_1 V_0 x \\ \Gamma_{\text{seq}} x \end{pmatrix}. \quad (252)$$

Hence S_{seq} is a transducer of the product $V = V_1 V_0$. The private block of S_{seq} is

$$T_{\text{seq}} = \begin{pmatrix} T_0 & 0 \\ B_1 E_0 & T_1 \end{pmatrix}. \quad (253)$$

Therefore,

$$(I - T_{\text{seq}})^{-1} = \begin{pmatrix} R_0 & 0 \\ R_1 B_1 E_0 R_0 & R_1 \end{pmatrix}, \quad (254)$$

where $R_j = (I - T_j)^{-1}$. Using the transducer identities $R_j B_j = \Gamma_j$ and $E_j R_j = V_j \Gamma_j^\dagger$, the off-diagonal block becomes $R_1 B_1 E_0 R_0 = \Gamma_1 V_0 \Gamma_0^\dagger$. Hence $\|R_1 B_1 E_0 R_0\| \leq \sqrt{w(S_1)w(S_0)}$. Consequently,

$$K(S_{\text{seq}}) = \|(I - T_{\text{seq}})^{-1}\| \leq \max\{K(S_0), K(S_1)\} + \sqrt{w(S_0)w(S_1)}. \quad (255)$$

Compared with the clock construction, the propagation term depends on the geometric mean of the catalyst complexities rather than the worst-case catalyst weight.

The above phenomenon also appears for $L \geq 3$. We briefly discuss the case $L = 3$. Now we introduce three private spaces and define

$$\hat{S}_0 = \begin{pmatrix} D_0 & E_0 & 0 & 0 \\ B_0 & T_0 & 0 & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & I \end{pmatrix}, \quad \hat{S}_1 = \begin{pmatrix} D_1 & 0 & E_1 & 0 \\ 0 & I & 0 & 0 \\ B_1 & 0 & T_1 & 0 \\ 0 & 0 & 0 & I \end{pmatrix}, \quad \hat{S}_2 = \begin{pmatrix} D_2 & 0 & 0 & E_2 \\ 0 & I & 0 & 0 \\ 0 & 0 & I & 0 \\ B_2 & 0 & 0 & T_2 \end{pmatrix}. \quad (256)$$

Set

$$S_{\text{seq}} = \hat{S}_2 \hat{S}_1 \hat{S}_0, \quad (257)$$

which corresponds to the block-encoding of $D_2 D_1 D_0$. The corresponding catalyst map is

$$\Gamma_{\text{seq}} x = \begin{pmatrix} \Gamma_0 x \\ \Gamma_1 V_0 x \\ \Gamma_2 V_1 V_0 x \end{pmatrix}. \quad (258)$$

Again, applying the defining relations of each transducer gives

$$S_{\text{seq}} \begin{pmatrix} x \\ \Gamma_{\text{seq}} x \end{pmatrix} = \begin{pmatrix} V_2 V_1 V_0 x \\ \Gamma_{\text{seq}} x \end{pmatrix}. \quad (259)$$

Thus the sequential construction realizes the desired product action. A similar calculation shows that the following bound

$$K(S_{\text{seq}}) \lesssim \max K(S_j) + \sqrt{w(S_0)w(S_1)} + \sqrt{w(S_1)w(S_2)} + \sqrt{w(S_0)w(S_2)w(S_1)}. \quad (260)$$

In summary, the sequential construction has a close connection with the standard product of block-encodings. This connection provides a natural Schur-complement interpretation of products of block encodings and reveals a multiplicative coupling structure in the resolvent norm $K(S)$. However, the resulting $K(S)$ bound can be worse due to the appearance of higher-order products of catalyst weights.

In contrast, the clock construction in Lemma 3.2 is motivated by the core idea of LCU: all components are coherently selected within an enlarged Hilbert space, while the intermediate computation is stored in the private space. Although this construction may lose some of the finer multiplicative structure of the sequential composition, it has important algorithmic advantages. Under the shared primitive-access assumption 2.1, the select oracle of the individual transducers can be implemented efficiently, so the whole composed transducer can be realized with a single coherent query to the underlying primitive oracles. Consequently, the action can be implemented with cost $\mathcal{O}(K(S) \log(1/\epsilon))$ described in Theorem 3.5. For the sequential construction, implementing the product requires applying each S_j individually, resulting in an additional factor depending on the composition length L .